

- B. 每个有向图都至少存在一个拓扑排序
C. 拓扑排序一定从入度为 0 的结点开始
D. 拓扑排序的方案一定是唯一的
10. 下列说法中哪个不是树的性质? ()
A. 任意两个结点之间有且只有一条简单路径
B. 树中有可能有一个简单环
C. 边的数目恰好是顶点数目减 1
D. 树中无环
11. 对于完全背包问题 (给出 n 种物品和一个容积为 m 的背包, 每种物品有无限个, 单个大小为 $v[i]$, 价值为 $w[i]$, 要求选择合适的物品放入背包, 满足大小不超过容积且价值最大), 设 $f[i]$ 表示用去 i 的空间能获得的最大价值, 倒序枚举 i 为使用的空间, 正序枚举 j 为物品编号, 则可写出动态转移方程 ()。
A. $f[i] = \max(f[i], f[i-w[j]] + v[j])$
B. $f[i] = \max(f[i], f[i-v[j]] + w[j])$
C. $f[i] = \min(f[i], f[i-w[j]] + v[j])$
D. $f[i] = \min(f[i], f[i-v[j]] + w[j])$
12. 前缀表达式 $**a+bcd$ 的后缀表达式是 ()。
A. $abcd**+$ B. $abc**d*$ C. $a*bc**d$ D. $b+c**d$
13. 某学校“信奥”小组有男生和女生各 3 名, 现从中挑选 2 名学生去参加市科技节编程竞赛, 至少有 1 名男生参加的概率是 ()。
A. 0.5 B. 0.6 C. 0.7 D. 0.8
14. 四面体的顶点及各棱中点加起来共有 10 个点, 在其中取 4 个不共面的点, 不同的取法共有 () 种。
A. 150 B. 147 C. 144 D. 141
15. 设全集 $E=\{a, b, c, d, e\}$, 集合 $A=\{a, c\}$, $B=\{a, b, d\}$, $C=\{b, d\}$, 则集合 $(A \cap B) \cup \sim C$ 为 ()。
A. $\{a\}$ B. $\{c, e\}$ C. $\{a, e\}$ D. $\{a, c, e\}$

二、阅读程序（程序输入不超过数组或字符串定义的范围；判断题正确填√，错误填×；除特殊说明外，判断题每题 1.5 分，选择题每题 3 分，共计 40 分）

(1)

```
01 #include<bits/stdc++.h>
02 using namespace std;
03
04 int dp[502][502];
05
06 int main() {
07     string s;
08     cin >> s;
09     for (int j=0; j<=s.size(); j++) {
10         for (int i = 0; i<=s.size()-j; i++) {
11             dp[i][i+j] = dp[i+1][i+j] + 1;
12             for (int k=i+1; k<=i+j; k++) {
13                 if (s[k] == s[i]) {
14                     dp[i][i+j]=min(dp[i][i+j],dp[i+1][k-1]+dp[k+1][i+j]);
15                 }
16             }
17         }
18     }
19     cout << dp[0][s.size()-1] << endl;
20     return 0;
21 }
```

■ 判断题

16. 将第 11 行改为 `dp[i][j]=INT_MAX-1`，代码的输出结果会改变。 ()
17. 修改第 13 行为 `if (s[i] == s[j])`，代码的输出结果不变。 ()
18. 用贪心算法可以实现与本程序同样的功能。 ()
19. 将第 12 行中 `for` 循环的 `k` 的初始值从 `i + 1` 改为 `i`，代码的输出结果不变。 ()

■ 选择题

20. 本程序的时间复杂度是 ()。
- A. $O(n^2)$ B. $O(n^3)$ C. $O(n^3 \log n)$ D. $O(n^4)$

21. 对于输入 aagog, 本程序的输出为 ()。

A. 1

B. 2

C. 3

D. 5

(2)

```
01 #include <bits/stdc++.h>
02
03 using namespace std;
04 pair<int, int> pi;
05
06 int parent[100000], sz[100000];
07
08 int n, m, components;
09
10 void initialize(){
11     for(int i=0; i<n; i++){
12         parent[i] = i;
13         sz[i] = 1;
14     }
15 }
16
17 int find(int a){
18     if(parent[a] == a){
19         return a;
20     }
21     return find(parent[a]);
22 }
23
24 bool sameDSU(int a, int b){
25     return find(a) == find(b);
26 }
27
28 void merge(int a, int b){
29     a = find(a), b = find(b);
30     if (a == b) return;
31     --components;
32     if (sz[a] > sz[b]) swap(a, b);
33     parent[a] = b;
34     sz[b] += sz[a];
35 }
36
```

```
37 int findsz(int a){
38     return sz[find(a)];
39 }
40
41 int main(){
42     cin >> n >> m;
43     initialize();
44     components = n;
45     int maxComponents = 1;
46     for(int i=0; i<m; i++){
47         int a, b; cin >> a >> b;
48         merge(a, b);
49         maxComponents = max(maxComponents, findsz(a));
50         cout << components << " " << maxComponents << endl;
51     }
52 }
```

■ 判断题

22. 如果输入的 a 和 b 从 1 开始编号，程序依然可以正常运行。 ()
23. 该程序计算了无向图的连通分量数。 ()
24. 合并操作总是将较小树的根结点合并到较大树的根结点下。 ()
25. 为了确保该程序不发生错误，需要保证输入的 a 和 b 互不相同。 ()

■ 选择题

26. 在程序运行结束时，maxComponents 变量可以存储的最大值是 ()。
- A. $n/2$ B. n C. $m/2$ D. m
27. 如果合并操作 merge 总是将结点 b 合并到结点 a，不论它们各自的 sz 大小如何，程序的最坏运行时间会如何变化? ()
- A. 变快 B. 保持不变 C. 变慢 D. 无法确定

(3)

```
01 #include<bits/stdc++.h>
02 using namespace std;
03 typedef long long ll;
04
05 const int MAXN = 2e5 + 7;
```

```
06 const int BITS = 30;
07 int numNodes, nodeValues[MAXN];
08 int trieNodeCount, trie[MAXN*BITS][2], maxIndex[MAXN*BITS];
09
10 void insert(int value, int index) {
11     int currentNode = 0;
12     for(int bit = BITS - 1; bit >= 0; bit--) {
13         int currentBit = (value >> bit) & 1;
14         if(!trie[currentNode][currentBit]) {
15             trie[currentNode][currentBit] = trieNodeCount++;
16         }
17         currentNode = trie[currentNode][currentBit];
18         maxIndex[currentNode]=max(maxIndex[currentNode],index);
19     }
20 }
21
22 int query(int value, int index) {
23     int currentNode = 0, result = 0;
24     for(int bit = BITS - 1; bit >= 0; bit--) {
25         int currentBit = (value >> bit) & 1;
26         if(trie[currentNode][currentBit] && maxIndex[trie[currentNode]
            [currentBit]] >= index) {
27             currentNode = trie[currentNode][currentBit];
28         } else {
29             currentNode = trie[currentNode][1 - currentBit];
30             result += (1 << bit);
31         }
32     }
33     return result;
34 }
35
36 ll weight;
37 void findMinimumXor(int left, int right, int bitPosition) {
38     if(left == right || bitPosition < 0) return;
39     int mid = left;
40     while(mid<right && !((nodeValues[mid]>>bitPosition)&1)) mid++;
41     findMinimumXor(left, mid, bitPosition - 1);
42     findMinimumXor(mid, right, bitPosition - 1);
43     if(mid == right || mid == left) return;
44     int minEdgeWeight = INT_MAX;
45     for(int i = left; i < mid; i++) {
```


三、完善程序（单选题，每小题 3 分，共计 30 分）

(1) 输入一个整数 N 和 N 个 $[0, 2^{21}-1]$ 范围内的整数，求出从任意位置开始的最大区间异或和。如果有多个这样的子区间，则选择最短子区间，输出区间的左右端点。

```
01 #include<bits/stdc++.h>
02 using namespace std;
03 const int MAXN = 1e5 + 10;
04 const int MAXM = MAXN * 21;
05 int n, x;
06 int s[MAXN], id[MAXM];
07 int tree[MAXM][2], idx;
08
09 void insert(int x, int k) {
10     int p = 0;
11     for (int i = 20; i >= 0; i--) {
12         int u = x >> i & 1;
13         if (!tree[p][u]) ①;
14         ②;
15     }
16     id[p] = k;
17 }
18
19 int query(int x) {
20     int p = 0;
21     for (int i = 20; i >= 0; i--) {
22         int u = x >> i & 1;
23         if ( ③ ) p = tree[p][u ^ 1];
24         else p = tree[p][u];
25     }
26     return id[p];
27 }
28
29 int main() {
30     cin >> n;
31     insert(s[0], 0);
32     int res = -1;
33     int l, r;
```

```
34   for (int i = 1; i <= MAXN; ++i) {
35       cin >> x;
36       s[i] = s[i - 1] ^ x;
37       int k = query(s[i]);
38       int maxn = ④;
39       if (maxn > res) {
40           res = maxn;
41           l = k + 1, r = i;
42       }
43       ⑤;
44   }
45   cout << res << " " << l << " " << r << endl;
46   return 0;
47 }
```

34. ①处应填 ()。

- A. tree[p][u] = ++index B. tree[p][u] = tree[p][u^1]
C. tree[p][u] = ++p D. tree[p^1][u] = ++index

35. ②处应填 ()。

- A. p = index B. p = u
C. p = tree[p][u] D. p = ++p

36. ③处应填 ()。

- A. tree[p][u] B. !tree[p][u^1]
C. !tree[p][u] D. tree[p][u^1]

37. ④处应填 ()。

- A. s[i]^s[k] B. s[r]^s[k]
C. s[i]^x D. s[l]^s[r]

38. ⑤处应填 ()。

- A. insert(s[i], i) B. insert(s[r], r-1)
C. insert(s[r], r) D. insert(s[i], i-1)

(2) 给定一个 $n \times m$ 的矩阵, 将 $r \times c$ 的矩阵分成 $r' \times c'$ 和 $r \times (c-c')$ 的矩阵需要 r^2 的代价。求分出一个大小和为 k 的矩阵所需的最小代价。

输入样例 1:

2 2 1

输出样例 1:

5

输入样例 2:

2 2 3

输出样例 2:

5

样例 1 解释: 一共需要进行 2 次操作。

- i. 把 2×2 的矩阵分为两个 2×1 的矩阵, 代价为 $2^2=4$ 。
- ii. 把 2×1 的矩阵分为两个 1×1 的矩阵, 代价为 $1^2=1$, 此时得到 1×1 的矩阵的和为 1, 总代价为 $1+4=5$ 。

样例 2 解释: 一共需要进行 2 次操作。

- i. 把 2×2 的矩阵分为两个 2×1 的矩阵, 代价为 $2^2=4$ 。
- ii. 把 2×1 的矩阵分为两个 1×1 的矩阵, 代价为 $1^2=1$, 此时得到 1×1 的矩阵和 2×1 的矩阵的和为 3, 总代价为 $1+4=5$ 。

```
01 #include<bits/stdc++.h>
02 using namespace std;
03 const int MAXN = 1e5 + 10;
04 const int MAXM = MAXN * 21;
05 int n, x;
06 int s[MAXN], id[MAXM];
07 int tree[MAXM][2], idx;
08
09 void insert(int x, int k) {
10     int p = 0;
11     for (int i = 20; i >= 0; i--) {
12         int u = x >> i & 1;
13         if (!tree[p][u]) ①;
14         ②;
15     }
16     id[p] = k;
17 }
18
```

```

19 int query(int x) {
20     int p = 0;
21     for (int i = 20; i >= 0; i--) {
22         int u = x >> i & 1;
23         if (③) p = tree[p][u ^ 1];
24         else p = tree[p][u];
25     }
26     return id[p];
27 }
28
29 int main() {
30     cin >> n;
31     insert(s[0], 0);
32     int res = -1;
33     int l, r;
34     for (int i = 1; i <= MAXN; ++i) {
35         cin >> x;
36         s[i] = s[i - 1] ^ x;
37         int k = query(s[i]);
38         int maxn = ④;
39         if (maxn > res) {
40             res = maxn;
41             l = k + 1, r = i;
42         }
43         ⑤;
44     }
45     cout << res << " " << l << " " << r << endl;
46     return 0;
47 }

```

39. ①处应填 ()。

A. $f[x-1][y-1][z-1]$

B. $f[x][y][z]$

C. $f[x\%2][y\%2][z\%2]$

D. $f[\text{MAXN}-x][\text{MAXN}-y][\text{MAXN}-z]$

40. ②处应填 ()。

A. $z == 1$

B. $z == x*y$

C. $z == x+y$

D. $z >= \text{MAXN}$

41. ③处应填 ()。

A. INF

B. -MAXN

C. 0

D. MAXN

42. ④处应填()。

A. $y \star y + \text{solve}(y, i, j) + \text{solve}(x-i, y, z-j)$

```
B.  $y \star y + \text{solve}(i, y, j) + \text{solve}(x-i, y, z-j)$ 
```

C. $(x-i) * (x-i) + \text{solve}(x-i, i, j) + \text{solve}(x-i, y, z)$

D. $(x-i) * (x-i) + \text{solve}(i, x-i, j) + \text{solve}(x, y-i, z-j)$

43. ⑤处应填()。

```
A.res = f[x][y][z]
```

```
B.f[x-1][y-1][z-1] = res
```

```
C.f[x][y][z] = res
```

```
D.res += f[x][y][z]
```

扫码获取
答案解析

提高组 CSP-S 2024 初赛模拟卷 2

一、单项选择题（共 15 题，每题 2 分，共计 30 分；每题有且仅有一个正确选项）

1. 在 NOI Linux 系统终端中，以下哪个命令可以用来创建一个新的空白文档？（ ）

- A. mkdir B. cp -a C. mv D. touch

2. 以下关于数据结构的表述中不恰当的一项是（ ）。

- A. 栈是一种后进先出的数据结构
B. 非连通有向图的边数一定小于顶点数
C. 队列是一种先进先出的数据结构
D. 哈希表是一种通过哈希函数将关键字映射到存储位置的数据结构

3. 对于 4 个结点的简单有向图，最少（ ）条边可以形成一个覆盖所有结点的环。

- A. 2 B. 3 C. 4 D. 5

4. 对于给定的正整数 a 和 n (n 为 2 的正整数次幂)，下面求 a^n 值的方法的最准确描述是（ ）。

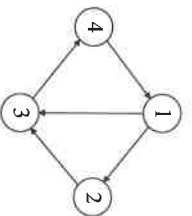
```
long long fun(int a, int n)
{
    long long ret = 1;
    if(n<=1)
        return pow(a, n);
    else
    {
        ret = fun(a, n/2);
        return ret*ret;
    }
}
```

- A. 倍增 B. 二分 C. 折半 D. 迭代

5. 约定杨辉三角形第 0 行只有 1 个元素是 1，第 1 行有 2 个元素是 1，则第 5 行的所有元素之和是（ ）。

- A. 8 B. 16 C. 32 D. 64

6. 下列哪个问题不能用贪心法精确求解? ()
- A. 哈夫曼编码 B. 多重背包 C. 打水问题 D. 最小生成树
7. 对于具有 n 个元素的二叉排序树 (又名二分查找树), 进行后序遍历的时间复杂度是 ()。
- A. $O(\log n)$ B. $O(n)$ C. $O(n^2)$ D. $O(n \log n)$
8. 考虑对 n 个数进行排序, 以下方法中最坏时间复杂度最优的排序方法是 ()。
- A. 选择排序 B. 快速排序 C. 堆排序 D. 插入排序
9. 下面有向图中的数字表示顶点序号, 则从 1 号顶点出发的 BFS 遍历输出的顶点序列可能是 ()。



- A. 1 4 3 2 B. 1 4 2 3 C. 1 3 2 4 D. 1 2 4 3
10. 给定地址区间为 0~10 的哈希表, 哈希函数为 $h(x) = x \% 11$, 采用线性探查的冲突解决策略 (若出现冲突情况, 会往后探查第一个空的地址存储; 若地址 10 冲突了, 则从地址 0 重新开始探查)。哈希表初始为空表, 依次存储 (60, 34, 62, 88, 22, 57, 78) 后, 78 存储在哈希表的哪个地址中? ()
- A. 1 B. 2 C. 3 D. 4
11. STL 中的容器可以分为顺序容器和关联容器。以下哪个不属于顺序容器? ()
- A. multiset B. vector C. list D. deque
12. 二分图是指能将顶点划分成两个部分, 且每一部分内的顶点间没有边相连的简单无向图。含有 24 个顶点的二分图 (两部分的顶点和之差不超过 6) 至少有 () 条边。
- A. 144 B. 128 C. 135 D. 140
13. 令二叉树根结点的高度为 0, 则一棵含有 2024 个结点的二叉树的高度可能是 ()。
- A. 2024 B. 2023 C. 9 D. 8

14. 设全集 $I = \{1, 2, 3, 4, 5\}$, 选择集合 I 的两个非空子集 A 和 B , 要使 B 中最小的数大于 A 中最大的数, 则不同的选择方法有 () 种。
- A. 50 B. 49 C. 48 D. 47
15. 设全集 $I = \{1, 2, 3, 4, 5, 6, 7, 8\}$, 集合 $B \cup A = \{1, 2, 3, 4, 5, 6\}$, $C \cap A = \{3, 4, 5\}$, $\sim B \cap A = \{1, 4\}$, 那么集合 $C \cap B \cap A$ 为 ()。
- A. $\{3, 5\}$ B. $\{4, 5\}$ C. $\{3, 4, 5\}$ D. $\{4, 6\}$

二、阅读程序 (程序输入不超过数组或字符串定义的范围; 判断题正确填√, 错误填×; 除特殊说明外, 判断题每题 1.5 分, 选择题每题 3 分, 共计 40 分)

(1)

```
01 #include <bits/stdc++.h>
02 using namespace std;
03 typedef pair <int,int> PII;
04 const int N = 1007;
05 int n,m,ans;
06 char g[N][N];
07 bool st[N][N];
08 void bfs(int x,int y) {
09     queue <PII> q;
10     q.push({x,y});
11     st[x][y] = true;
12     while(q.size()) {
13         PII t = q.front();
14         q.pop();
15         for (int i=t.first-1; i<=t.first+1; ++i)
16             for (int j=t.second-1; j<=t.second+1; ++j) {
17                 if (i==t.first && j==t.second)
18                     continue;
19                 if (i < 1 || i > n || j < 1 || j > m)
20                     continue;
21                 if (g[i][j]!='W' || st[i][j])
22                     continue;
23                 q.push({i,j});
24                 st[i][j]=true;
25             }
26     }
27 }
```

```
28 int main()
29 {
30     cin >> n >> m;
31     for(int i = 1; i <= n; ++i)
32         scanf("%s", g[i]+1);
33     for(int i = 1; i <= n; ++i)
34         for(int j = 1; j <= m; ++j)
35             if(g[i][j] == 'W' && !st[i][j]) {
36                 bfs(i, j);
37                 ++ans;
38             }
39     printf("%d\n", ans);
40     return 0;
41 }
```

■ 判断题

16. 若将第 30 行改为 `cin >> m >> n;`, 程序的输出不变。 ()
17. 该问题也可以用 DFS 解决。 ()
18. 当输入为
- ```
4 5
WSWWSS
WWWSSS
SSWSWS
SSSSWS
```
- 时, 程序会崩溃。 ( )
19. 从程序中可以看出题目允许的拓展方向有 4 个。 ( )

#### ■ 选择题

20. 对于输入

```
4 5
WSWWS
WWWSS
SSWSW
SSSSW
```

程序的输出是 ( )。

- A. 1                      B. 2                      C. 3                      D. 4

21. 本程序的时间复杂度是 ( )。

A.  $O(n)$

B.  $O(n^2m^2)$

C.  $O(n^2m)$

D.  $O(nm)$

(2)

```
01 #include <bits/stdc++.h>
02 using namespace std;
03
04 typedef struct {
05 double x;
06 double y;
07 } Point;
08
09 Point point_init(double x, double y) {
10 Point p;
11 p.x = x;
12 p.y = y;
13 return p;
14 }
15
16 Point point_sub(Point a, Point b) {
17 Point p;
18 p.x = a.x - b.x;
19 p.y = a.y - b.y;
20 return p;
21 }
22
23 double cross(Point a, Point b) {
24 return a.x * b.y - a.y * b.x;
25 }
26
27 double polygonArea(const vector<Point>& v) {
28 double area = cross(v.back(), v.front());
29 for (size_t i=0; i<v.size()-1; ++i) {
30 area += cross(v[i], v[i+1]);
31 }
32 return area;
33 }
34
35 void solve(int n) {
36 vector<Point> pts(n);
37 for (int i = 0; i < n; ++i) {
38 double a, b;
```

```
39 cin >> a >> b;
40 pts[i] = point_init(a, b);
41 }
42 double area = polygonArea(pts) / 2;
43 cout<<fixed<<setprecision(1)<<fabs(area)<<"\n";
44 }
45
46 int main() {
47 while (true) {
48 int n;
49 cin >> n;
50 if (n == 0) break;
51 solve(n);
52 }
53 return 0;
54 }
```

#### ■ 判断题

22. 将第 42 行改为 `double area = polygonArea(pts) >> 1;`, 程序正常运行。 ( )
23. 无论点是按逆时针方向还是按顺时针方向给出, 程序都可以计算出正确的结果。 ( )
24. 程序可以处理凹多边形而不需要任何修改。 ( )
25. `polygonArea` 的返回值始终为正。 ( )

#### ■ 选择题

26. `cross` 函数的计算结果用于表示 ( )。
- A. 两个点的欧几里得距离                      B. 连接两点的直线的斜率
- C. 两个向量的点积                              D. 两个向量的叉积

27. 对于输入

```
3
1 1
0 0
1 0
```

程序输出 ( )。

- A. 1.0                      B. 0.5                      C. 0.0                      D. 0.6

## (3)

```
01 #include <bits/stdc++.h>
02 using namespace std;
03
04 class BIT {
05 public:
06 vector<int> bit;
07 int size;
08 BIT(int n) : size(n), bit(n + 1, 0) {}
09 void update(int x, int delta) {
10 ++x;
11 while(x <= size) {
12 bit[x] += delta;
13 x += x & -x;
14 }
15 }
16 int query(int x) const {
17 ++x;
18 int sum = 0;
19 while(x > 0) {
20 sum += bit[x];
21 x -= x & -x;
22 }
23 return sum;
24 }
25 };
26
27 int main() {
28 int n, q;
29 cin >> n >> q;
30 vector<int> arr(n);
31 vector<pair<int, int> > queries[q];
32 vector<int> solution(q);
33 for (int i = 0; i < n; ++i) {
34 cin >> arr[i];
35 }
36 for (int i = 0; i < q; ++i) {
37 int a, b;
38 cin >> a >> b;
39 a--, b--;
40 queries[a].emplace_back(b, i);
```

```

41 }
42 BIT bit(n);
43 map<int, int> last_index;
44 for (int i = n - 1; i >= 0; --i) {
45 int val = arr[i];
46 if (last_index.find(val) != last_index.end()) {
47 bit.update(last_index[val], -1);
48 }
49 last_index[val] = i;
50 bit.update(i, 1);
51
52 for (const auto& query: queries[i]) {
53 solution[query.second] = bit.query(query.first);
54 }
55 }
56 for (int ans : solution) {
57 cout << ans << ' ';
58 }
59 return 0;
60 }

```

### ■ 判断题

28. 程序支持在线查询。 ( )
29. 将第 13 行改为 `x += x & ( ~x + 1 );`, 程序功能不变。 ( )

### ■ 选择题

30. BIT 类中的 query 函数的返回值是 ( )。
- A. 指定索引的值
  - B. 自根开始到指定索引的所有值的乘积
  - C. 指定索引之前所有值的累加和
  - D. 自根开始到指定索引的所有值的最大值
31. 程序的时间复杂度为 ( )。
- A.  $O(n)$
  - B.  $O(n \log n)$
  - C.  $O(n^2 \log n)$
  - D.  $O(\log n)$
32. 如果输入的 arr 数组的元素大小互不相同, 则 last\_index.size() 的值是 ( )。
- A. 0
  - B. 1
  - C. n
  - D. q

33. (4分) 如果输入为

```
5 3
3 2 3 1 2
1 3
2 4
1 5
```

则输出将是 ( )。

- A. 2 3 3      B. 3 2 4      C. 3 2 5      D. 0 1 0

### 三、完善程序 (单选题, 每小题 3 分, 共计 30 分)

(1) 定义\*数字, 对于一个没有前导 0 的数, 数字  $d$  恰好出现在这个数的所有偶数位置。  
现在给定  $m, d, a, b$ , 询问  $[a, b]$  范围内是  $m$  倍数的\*数字的数量, 答案对  $1e9+7$  取模。

```
01 #include <bits/stdc++.h>
02 using namespace std;
03
04 const int M = 2005;
05 const int mod = 1e9+7;
06 int dp[M][M][2];
07 int m,d;
08
09 int dfs(int i, int sum, int small_taken, string ss){
10 if(①) return sum == 0;
11 int ans = dp[i][sum][small_taken];
12 if(ans != -1) return ans;
13 ans = 0;
14 int digit = s[i]-'0';
15 int low = 0, high = digit;
16 if(small_taken) high = 9;
17 for(int j = low; j <= high; ++j){
18 if(i % 2 == 1 && j != d) continue;
19 if(i % 2 == 0 && j == d) continue;
20 int new_small = ②;
21 int new_sum = ③;
22 ans += dfs(i + 1, new_sum , new_small, s);
23 ans %= mod;
24 }
}
```

```
25 return dp[i][sum][small_taken] = ans;
26 }
27
28 int main(){
29 string a,b;
30 cin >> m >> d >> a >> b;
31 int is_a_magic = 1;
32 int res = 0;
33 for(int i = 0; i < a.size(); ++i){
34 int dig = a[i] - '0';
35 if(i % 2 == 0 && dig == d) is_a_magic = 0;
36 if(④) is_a_magic = 0;
37 res = (res * 10 + dig) % m;
38 }
39 if(res) is_a_magic = 0;
40 memset(dp,-1,sizeof dp);
41 int sum_a = dfs(0,0,0,a);
42 memset(dp,-1,sizeof dp);
43 int sum_b = dfs(0,0,0,b);
44 cout << ⑤ << endl;
45 return 0;
46 }
```

34. ①处应填 ( )。

- A.  $i \geq s.size()$                       B.  $i > s.size()$   
C.  $i \geq sum$                               D.  $i > sum$

35. ②处应填 ( )。

- A.  $j! = d \ || \ j < high$                       B.  $small\_taken \ \&\& \ j < high$   
C.  $j == d \ || \ j < high$                       D.  $small\_taken \ || \ j < high$

36. ③处应填 ( )。

- A.  $(sum + dig) \% m$                       B.  $(sum * 10 + digit) \% m$   
C.  $(sum + high) \% m$                       D.  $(sum * 10 + high) \% m$

37. ④处应填 ( )。

- A.  $i \% 2 == 1 \ \&\& \ dig != d$                       B.  $dig != d \ || \ is\_a\_magic$   
C.  $i \% 2 == 0 \ || \ dig != d$                       D.  $dig != d \ \&\& \ !is\_a\_magic$

38. ⑤处应填 (        )。

- A.  $(\text{sum\_b} - \text{sum\_a} + \text{is\_a\_magic} + \text{mod}) \% \text{mod}$
- B.  $(\text{sum\_b} - \text{sum\_a} + \text{is\_a\_magic}) \% \text{mod}$
- C.  $(\text{sum\_b} - \text{sum\_a} + \text{!is\_a\_magic} + \text{mod}) \% \text{mod}$
- D.  $(\text{sum\_b} - \text{sum\_a} + \text{!is\_a\_magic}) \% \text{mod}$

(2) 给定  $n$  个人和  $m$  条信息，每条信息描述了若干人的排队要求，比如一条信息是 2、5、1，那么排队时 2 号应该在 5 号前，5 号应该在 1 号前。这些信息是按照优先级排列的，所以现在希望你能找到一个排队序列满足前  $X$  条信息，使得这个  $X$  尽可能大。对于这个最大的  $X$ ，你需要找到字典序最小的答案。

输入格式：

第 1 行包含  $N$  和  $M$ 。接下来的  $M$  行，每行描述了一条信息，第  $i+1$  行描述了第  $i$  条信息，第一个数是本条信息中涵盖的人员数量  $\text{num}_i$ ，后面是一列  $\text{num}_i$  个整数。

输出格式：

输出使得  $X$  最大的最小字典序排队序列。

输入样例：

```
4 3
3 1 2 3
2 4 2
3 3 4 1
```

输出样例：

```
1 4 2 3
```

```
01 #include <bits/stdc++.h>
02 #define LL long long
03 #define PII pair<int,int>
04 using namespace std;
05
06 vector<int> ret;
07 vector<vector<int>> > order;
08 int n, m;
09
10 struct TopoSort {
11 int N;
12 vector<int> in, res;
13 vector<vector<int>> > adj;
```

```
14 void init(int _N) {
15 N = _N;
16 in.resize(N);
17 adj.resize(N);
18 }
19 void addedge(int x, int y) { adj[x].push_back(y), ①; }
20 bool sort() {
21 ②;
22 for (int i = 1; i < N; i++)
23 if (③) todo.push(i);
24 while (todo.size()) {
25 int x = todo.top();
26 todo.pop();
27 res.push_back(x);
28 for (const int &i : adj[x])
29 if (!(--in[i])) todo.push(i);
30 }
31 return res.size() == N - 1;
32 }
33 };
34
35
36 bool check(int x) {
37 TopoSort g;
38 g.init(n + 1);
39 for (int i = 0; i < x; i++) {
40 for (int j = 0; j < order[i].size() - 1; j++) {
41 g.addedge(order[i][j], order[i][j + 1]);
42 }
43 }
44 bool ans = g.sort();
45 if(ans)
46 ④;
47 return ans;
48 }
49
50 int main() {
51 cin >> n >> m;
52 order.resize(m);
53 for (int i = 0; i < m; i++) {
54 int k;
55 cin >> k;
56 vector<int> v(k);
57 for (int j = 0; j < k; j++) cin >> v[j];
```

```
58 order[i] = v;
59 }
60
61 int l = 0, r = m;
62 while (l < r) {
63 int mid = ⑤;
64 if(check(mid))
65 l = mid;
66 else
67 r = mid - 1;
68 }
69 for (int i = 0; i < n; i++) {
70 cout << ret[i] << " ";
71 }
72 return 0;
73 }
```

39. ①处应填 ( )。

- A. ++in[x]                      B. adj[y].push\_back(x)  
C. ++in[y]                      D. ++res[x]

40. ②处应填 ( )。

- A. priority\_queue<int, vector<int>, greater<int> > todo  
B. priority\_queue<int, vector<int>, less<int> > todo  
C. set<int, less<int> > todo  
D. set<int, greater<int> > todo

41. ③处应填 ( )。

- A. in[i] == i    B. in[i]                      C. in[i] != i    D. !in[i]

42. ④处应填 ( )。

- A. ret[x] = g.res[x]  
B. ret = g.res, ret.resize(x)  
C. ret.resize(x), ret[x] = g.res[x]  
D. ret = g.res

43. ⑤处应填 ( )。

- A. (l + r) >> 1                      B. l + (r - l + 1) >> 1  
C. (l + r) / 2                      D. l + (r - l + 1) / 2

## 提高组 CSP-S 2024 初赛模拟卷 3

扫码获取  
答案解析

一、单项选择题（共 15 题，每题 2 分，共计 30 分；每题有且仅有一个正确选项）

1. 下面的代码属于哪个算法的核心代码？（ ）

```
long long c[20][20];
c[0][0]=1;
for(int i=1;i<=n;i++){
 c[i][0]=c[i][i]=1;
 for(int j=1;j<i;j++)
 c[i][j]=c[i-1][j-1]+c[i-1][j]
}
```

- A. 0-1 背包      B. 杨辉三角形      C. 完全背包      D. 部分背包

2. NOI Linux 2.0 中缺省指定编译的 C++ 版本是（ ）。

- A. C++03      B. C++17      C. C++11      D. C++14

3. 下列属于视频文件格式的是（ ）。

- A. PNG      B. AVI      C. JPEG      D. BMP

4. 以下不属于常见的平衡树的是（ ）。

- A. AVL 树      B. 伸展树      C. 二叉树      D. 红黑树

5. 字符串 "dededfed" 和字符串 "defed" 的最长公共子串是（ ）。

- A. fed      B. defed      C. def      D. de

6. 下面哪个操作不属于二叉搜索树的常见操作？（ ）

- A. 查找      B. 插入      C. 删除      D. 交换排序

7. 具有  $m$  个顶点、 $n$  条边的有向图采用邻接表存储结构，进行深度优先遍历运算的时间复杂度是（ ）。

- A.  $O(m)$       B.  $O(n)$       C.  $O(m+n)$       D.  $O(mn)$

8. 将关键字集合 {100, 300, 800, 700, 500, 900} 逐一保存在一张长度为 100 的哈希表中。选取哈希函数为  $\text{Hash}(\text{key}) = \text{key} / 100$ ，则 500 保存在表中的位置应该是 ( )。
- A. 5                      B. 6                      C. 7                      D. 8
9. 设树 T 的高度为 4，其中度为 1、2、3 和 4 的结点个数分别为 4、2、1、1，则 T 中的叶子结点数为 ( )。
- A. 5                      B. 6                      C. 7                      D. 8
10. 在 C++ 程序中执行 `cout << (9>8>7>6)` 会输出什么结果? ( )
- A. 1                      B. 0                      C. true                      D. false
11. 已知 240 个一模一样的小球里面有一个次品，比正常球轻一些。现在有一架天平（每一端都可以放下 240 个小球），请问最少称重几次可以找出这个次品小球? ( )
- A. 6                      B. 4                      C. 5                      D. 7
12. 正六边形的中心和顶点共 7 个点，以其中 3 个点为顶点的三角形共有 ( ) 个。
- A. 30                      B. 35                      C. 32                      D. 28
13. 下面的 C++ 代码运行后，输出的是 ( )。
- ```
#include <bits/stdc++.h>
using namespace std;
int main()
{
    int a[100]={1,2};
    string s="CSP-S";
    cout<<s[a[2]]<<endl;
    return 0;
}
```
- A. C B. S C. P D. CSP-S
14. 由数字 0, 1, 2, 3, 4, 5 组成没有重复数字的六位数，其中个位数字小于十位数字的共有 () 个。
- A. 210 B. 300 C. 464 D. 600

15. 已知元素(7, 25, 14, 87, 51, 90, 5, 19, 20), 问这些元素以怎样的顺序进入栈, 才能使出栈的顺序满足: 7 在 51 的前面; 90 在 87 的后面; 20 在 14 的后面; 25 在 5 的前面; 19 在 90 的后面? ()

- A. 20, 5, 7, 51, 90, 25, 14, 19, 87
 B. 51, 5, 19, 20, 14, 8, 87, 90, 25
 C. 19, 20, 90, 7, 5, 25, 51, 14, 87
 D. 5, 25, 51, 7, 20, 19, 90, 87, 14

- 二、阅读程序 (程序输入不超过数组或字符串定义的范围; 判断题正确填√, 错误填×; 除特殊说明外, 判断题每题 1.5 分, 选择题每题 3 分, 共计 40 分)

(1)

```

01 #include <iostream>
02 using namespace std;
03
04 const int mod = 1e9+7;
05 const int N = 1010, C = 10010;
06 int nn, cc;
07 long long a[2][C], b[2][C];
08
09 int main() {
10     cin >> nn >> cc;
11     a[1][0] = 1; b[1][0] = 1;
12     for(int n = 2; n <= nn; n++) {
13         for(int c=min(cc,(n-1)*(n-2)/2)+1;
14             c<=min(cc,n*(n-1)/2); c++) {
15             b[(n-1) & 1][c] = b[(n-1) & 1][c-1];
16         }
17         a[n & 1][0] = 1; b[n & 1][0] = 1;
18         for (int c = 1; c <= min(cc, n*(n-1)/2); c++) {
19             int en = max(c-n+1, 0);
20             a[n & 1][c] = b[(n-1) & 1][c];
21             if (en > 0) a[n & 1][c] -= b[(n-1) & 1][en-1];
22             a[n & 1][c] %= mod;
23             if (a[n & 1][c] < 0) a[n & 1][c] += mod;
24             b[n & 1][c] = b[n & 1][c-1] + a[n & 1][c];
25         }

```

```

26     }
27     cout << a[ln & 1][cc] << endl;
28     return 0;
29 }

```

■ 判断题

16. 输入为"10 1"时, 输出为"10"。 ()
17. 输入为"4 3"时, 输出为"6"。 ()
18. 本题与计算逆序对有关。 ()
19. 第 16 行可以放到第 13 行前面, 程序的运行结果不会改变。 ()

■ 选择题

20. 本程序的时间复杂度是 ()。
- A. $O(n^2)$ B. $O(n^3)$ C. $O(n^2 \log n)$ D. $O(n^4)$

21. 数组 a、b 的调用经常有"81"的操作, 这是为了 ()。

- A. 没什么意义, 干扰阅读 B. 用位运算快速减一
- C. 快速获取二进制尾数 D. &运算符用于获取存储地址

(2)

```

01 #include <iostream>
02 #include <iomanip>
03 #include <math.h>
04 using namespace std;
05 pair<long long, long long> calc(int n, long long m){
06     if (n==0) return make_pair(0,0);
07     long long len = 1ll << (n-1), cnt = 1ll << (2*n-2);
08     pair<long long, long long> pos = calc(n-1, m % cnt);
09     long long x = pos.first, y = pos.second;
10     long long z = m / cnt;
11     if (z==0) return make_pair(y, x);
12     if (z==1) return make_pair(x, y+len);
13     if (z==2) return make_pair(x+len, y+len);
14     if (z==3) return make_pair(2*len-y-1, len-x-1);
15 }
16 int main() {
17     int n, p, q;
18     cin >> n >> p >> q;

```

```
19     cout << setprecision(3)
20
21     << sqrt(pow(1.0*calc(n, p).first-calc(n,q).first,2.0))
22         + pow(1.0*calc(n,p).second-calc(n,q).second,2.0))
23     << endl;
24     return 0;
25 }
```

■ 判断题

22. 本程序编译会报 warning。 ()
23. 若想程序不出错，p、q 不能超过 2^{2^n} 。 ()
24. 输入数据符合要求时，本程序的时间复杂度是 $O(\max\{N, \log M\})$ 。 ()

■ 选择题

25. 若输入为 "2 0 10"，输出为 ()。
- A. 4.243 B. 3.00 C. 4.24 D. 3.606
26. 当 $|p-q|=1$ 时，不论输入的 n 是多少，只要 p、q 在符合要求的范围内，则输出为 ()。
- A. 1 B. 2 C. -1 D. 0
27. 对同一个 n ，共有几种输入，可以让输出达到最大？ ()
- A. 2 B. 4 C. 1 D. 8

(3)

```
01 #include <iostream>
02 using namespace std;
03
04 const int N = 500010;
05 int n;
06 long long ans, mod = 1000000007;
07 long long a[N], b[N], A[N], B[N], sa[N], sb[N];
08
09 int main() {
10     cin >> n;
11     for (int i = 1; i <= n; i++) {
12         cin >> a[i];
13         A[i] = A[i-1] + a[i];
```

```
14 A[i] %= mod;
15 sa[i] = sa[i-1] + A[i];
16 sa[i] %= mod;
17 }
18 for (int i = 1; i <= n; i++) {
19     cin >> b[i];
20     B[i] = B[i-1] + b[i];
21     B[i] %= mod;
22     sb[i] = sb[i-1] + B[i];
23     sb[i] %= mod;
24 }
25 long long t;
26 for (int i = 0; i <= n; i++) {
27     t = A[i];
28     t *= B[i]; t %= mod;
29     t *= n; t %= mod;
30     ans += t;
31     ans %= mod;
32 }
33 for (int i = 0; i < n; i++) {
34     t = A[i] * (sb[n] - sb[i]);
35     t %= mod;
36     ans -= t;
37     if (ans < 0) ans += mod;
38     t = B[i] * (sa[n] - sa[i]);
39     t %= mod;
40     ans -= t;
41     ans %= mod;
42     if (ans < 0) ans += mod;
43 }
44 cout << ans << endl;
45 return 0;
46 }
```

■ 判断题

28. 因为有第 39 行，所以若去掉第 35 行，程序结果不受影响。 ()
29. 去掉第 37 行，程序结果不受影响。 ()
30. 第 34 行有溢出风险，必须按照第 27~28 行的方式来写。 ()

■ 选择题

31. 若输入 3 2 3 4 3 4 5, 则输出为 ()。
- A. 243 B. 244 C. 253 D. 254

32. 本题实际在计算 ()。

$$\begin{array}{ll} \text{A. } \sum_{r=1}^n \sum_{i=r}^n (\sum_{i=l}^r a_i * b_i) & \text{B. } \sum_{l=1}^n \sum_{r=l}^n (\sum_{i=l}^r a_i * b_i) \\ \text{C. } \sum_{l=1}^n \sum_{r=l}^n (\sum_{i=l}^r a_i * \sum_{i=l}^r b_i) & \text{D. } \sum_{l=1}^n \sum_{r=l}^n (\sum_{i=l}^r a_i * \sum_{i=l}^r b_i) \end{array}$$

33. (4 分) 本代码将原问题的时间复杂度从 $O(N^3)$ 优化到 $O(N)$ 。如果希望继续优化, 应该 ()。

- A. 没有太大优化空间
B. 减少取模运算
C. 将 long long 改为 int, 以提高处理速度
D. 做更多的预处理

三、完善程序 (单选题, 每小题 3 分, 共计 30 分)

- (1) 有一个字符串 S, 最开始其中只有一个字符 a, 之后你要对这个字符串进行若干次操作, 每次将其中每一个字符 c 替换成某个字符串 s (例如对于字符串 ball, 将其中的 l 替换为 na 后将得到 banana)。现在给定 l, r , 你需要输出变换完后, S 的第 1 个字符到第 r 个字符对应的子串是什么。

算法是逆序建树, 从最后一次操作开始, 将字符替换的规则视为树的连线, 叶子结点为最后被替换的字母。建树完毕后, 再根据 l, r 的数值, 在树上 DFS 遍历并输出。

```
01 #include <bits/stdc++.h>
02 using namespace std;
03 #define ll long long
04 const int NN = 2e5+10;
05 char c[NN]; string s[NN];
06 int first[26]; ll tot[NN], tf[26];
07 vector<ll> cnt[NN]; vector<int> nxt[NN];
08
09 int num(char c){
10     return ①;
11 }
```

```
12 ll search(int cur, ll l, ll r, ll acc){
13     for(int i = 0; i < cnt[cur].size(); i++){
14         if(acc >= r) break;
15         if(acc + cnt[cur][i] >= l){
16             if(!next[cur][i]){
17                 cout << s[cur][i];
18                 acc++;
19             }
20             else
21                 ②;
22         }
23         else
24             acc += cnt[cur][i];
25     }
26     return acc;
27 }
28 int main(){
29     ios_base::sync_with_stdio(false);
30     cin.tie(NULL);
31     ll l, r; int n;
32     cin >> l >> r >> n;
33     for(int i = 1; i <= n; i++){
34         cin >> c[i] >> s[i];
35     }
36     for(int i = n; i > 0; i--){
37         for(int j = 0; j < s[i].size(); j++){
38             ③;
39             first[num(c[i])] = i;
40         }
41         for(int i = 0; i < 26; i++) tf[i] = 1;
42         for(int i = n; i > 0; i--){
43             for(int j = 0; j < s[i].size(); j++){
44                 ④;
45                 cnt[i].push_back(tf[k]);
46                 tot[i] += tf[k];
47                 if(tot[i] > r) break;
48             }
49             tf[num(c[i])] = tot[i];
50         }
51         ⑤;
52 }
```

34. ①处应填 ()。

- A. c-a B. c-'a' C. c-'a'+1 D. 'c'-'a'

35. ②处应填 ()。

- A. acc = search(acc+next[cur][i], l, r, acc)
B. acc = search(next[cur][i], l-cnt[cur][i], r, acc)
C. acc = search(next[cur][i], l, r, acc)
D. acc = search(next[cur][i], l, r, acc+cnt[cur][i])

36. ③处应填 ()。

- A. next[i].push_back(first[num(s[i][j])])
B. next[i].push_back(num(s[i][j]))
C. cnt[i].push_back(num(s[i][j]))
D. cnt[i].push_back(first[num(s[i][j])])

37. ④处应填 ()。

- A. int k = 0 B. int k = num(c[i])
C. int k = num(s[i][j]) D. int k = first[num(s[i][j])]

38. ⑤处应填 ()。

- A. search(first[1], l, r, next[0])
B. search(first[1], l, r, 0)
C. search(0, l, r, 1)
D. search(1, l, r, 0)

(2) 给定函数 $h(x)$, $x \in Z$, $1 \leq x \leq n$, 定义 $\text{seepairs}(h)$ 为这样的无序数对 (i, j) 的个数: $i < j$, 且连接 $(i, h(i))$ 与 $(j, h(j))$ 的线段不低于 $(k, h(k))$, 其中 $i < k < j$ 。对 h 函数进行 Q 次调整, 每次把 $h(x)$ 调整为 $h(x) + y$, $y > 0$ 。求每次调整之后的 $\text{seepairs}(h)$ 。
数据规模: $n, Q \leq 2000$, 初始 $h(x)$ 和调整后的 $h(x)$ 都不超过 10^9 。

本代码使用 set 存储, 初始化每个 $h(i)$ 能“看到”的点的数量 $\text{see}(i)$, 则对 $\text{see}(i)$ 求和即为 $\text{seepairs}(h)$, 每次调整 $h(x)$ 之后, 因为 $h(x)$ 只增不减, 所以 $\text{see}(x)$ 只增不减。对于 $i > x$, $\text{see}(i)$ 不变, 对于 $i < x$, $\text{see}(i)$ 只减不减, 因此把 $\text{see}(1)$ 至 $\text{see}(x)$ 调整之后再求和, 即为调整后的 $\text{seepairs}(h)$ 。

```
01 #include<bits/stdc++.h>
02 using namespace std;
03
04 const int N = 2010;
05 int n, q, x, y; long long h[N];
06 set<int> see[N];
07 typedef set<int>::iterator iter;
08
09 bool small(int i, int k, int j) {
10     return ①;
11 }
12 int main() {
13     cin >> n;
14     for (int i = 1; i <= n; i++)
15         cin >> h[i];
16     for (int i = 1; i < n; i++) {
17         see[i].insert(i+1);
18         for (int j = i+2; j <= n; j++) {
19             ②;
20             see[i].insert(j);
21         }
22     }
23     cin >> q;
24     for (int qq = 1; qq <= q; qq++) {
25         cin >> x >> y;
26         h[x] += y;
27         for (int i = 1; i < x; i++) {
28             iter it = ③;
29             if (it == see[i].end()) {
30                 --it;
31             }
32             if (small(i, *it, x)) see[i].insert(x);
33             else {
34                 bool ins = false;
35                 if (*it == x) {
36                     ins = true;
37                     it ++;
38                 } else {
39                     --it;
40                     if (small(i, *it, x)) {
41                         see[i].insert(x);
42                     }
43                 }
44             }
45         }
46     }
47 }
```

```
41     ins = true;
42     it++;
43     ④;
44     }
45     }
46     if (ins)
47         while ( ⑤ ) {
48             it = see[i].erase(it);
49         }
50     }
51     }
52     if (x < n) {
53         see[x].clear();
54         see[x].insert(x+1);
55         for (int j = x+2; j <= n; j++) {
56             if (small(x, *see[x].rbegin(), j))
57                 see[x].insert(j);
58         }
59     }
60     int ans = 0;
61     for (int i = 1; i < n; i++)
62         ans += see[i].size();
63     cout << ans << endl;
64 }
65     return 0;
66 }
```

39. ①处应填 ()。

- A. $(h[k] - h[i]) * (j - k) < (h[j] - h[k]) * (k - i)$
- B. $(h[k] - h[i]) * (j - k) <= (h[j] - h[k]) * (i - k)$
- C. $(h[k] - h[i]) * (j - i) < (h[j] - h[i]) * (k - i)$
- D. $(h[k] - h[i]) * (j - i) <= (h[j] - h[i]) * (k - i)$

40. ②处应填 ()。

- A. `if (small(i, *see[i].rbegin(), j))`
- B. `if (small(i, *see[i].rbegin(), j-1))`
- C. `if (small(i, *see[i].end(), j))`
- D. `if (small(I, *see[i].end()-1, j-1))`

41. ③处应填 ()。
- A. `see[i].upper_bound(x)`
 - B. `see[i].lower_bound(x)`
 - C. `see[i].lower_bound(x)+1`
 - D. `see[i].upper_bound(x+1)`
42. ④处应填 ()。
- A. `it--`
 - B. `it++`
 - C. `//无内容`
 - D. `}`
43. ⑤处应填 ()。
- A. `it != see[i].end()-1 && !small(i, x, *(it))`
 - B. `it != see[i].end()-1 && !small(i, x, it)`
 - C. `it != see[i].end() && !small(i, x, *(it))`
 - D. `it != see[i].end() && !small(i, x, it)`

提高组 CSP-S 2024 初赛模拟卷 4

扫码获取
答案解析

一、单项选择题（共 15 题，每题 2 分，共计 30 分；每题有且仅有一个正确选项）

1. 运行如下代码的输出结果是（ ）。

```
char c = '0' + ' ' ;  
cout << c << endl;
```

- A. 0 B. P C. 48 D. 80

2. 以下代码段的功能是（ ）。

```
#include <iostream>  
using namespace std;  
int lowbit(int x) {  
    return (x & -x);  
}  
  
int main() {  
    int n, res = 0;  
    cin >> n;  
    while (n)  
        n -= lowbit(n), res++;  
    cout << res;  
    return 0;  
}
```

- A. 求该数转为二进制数以后各位上 1 的总数
B. 求该数转为二进制的反码
C. 求该数转为二进制的补码
D. 求该数转为二进制的原码

3. 前缀表达式 $- * + 1 \ 3 \ 4 \ 5 / 5 \ 6 \ 7$ 的后缀表达式为（ ）。

- A. $1 + 3 \ 4 * 5 - 5 \ 6 / 7$
B. $- * + 1 \ 3 \ 4 \ 5 / 5 \ 6 \ 7$
C. $1 \ 3 \ 4 + 5 * 5 \ 6 \ 7 / -$
D. $1 \ 3 \ 4 \ 5 * + 5 \ 6 \ 7 / -$

5. 以下哪个操作不属于 STL 中双端队列 (deque) 的操作函数: ()
- A. front B. back C. top D. erase

A. Knuth
B. McDonald
C. Morris
D. Pratt

A. $\neg P \vee Q$ B. $P \wedge Q$
C. $\neg(P \vee Q)$ D. $\neg(\neg Q \wedge P)$

O				
				X
	X			
			X	

A.10 B.11 C.12 D.13

A. 欧拉路有一个奇点

B. 欧拉回路有两个奇点

C. 对于一个无向图, 如果任意一对顶点都是连通的, 则此图被称为连通图

D. 不存在割点的无向图称为“2-边连通图”

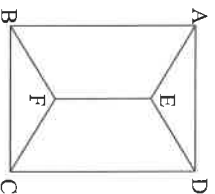
10. 假设我们用 $V = (p_1, p_2, \dots, p_5)$ 来表示无向图 G 的 5 个顶点的度数，下面给出的哪组 V 值合理？ ()
- A. {5, 3, 3, 3, 1} B. {2, 2, 2, 1, 1}
- C. {3, 3, 3, 2, 2} D. {1, 4, 3, 2, 5}

11. 下述选项哪个不属于与典型图论有关的算法？ ()
- A. 线段树 B. 次小生成树
- C. 最小生成树 D. Bellman-Ford

12. 有如下递归代码，则 $\text{solve}(30, 30)$ 的结果为 ()。
- ```
int fun(int a, int b){
 if(a == 1) return 1;
 return 9 * fun(a - 1, b) % b;
}
```

- A. 9                                          B. 18                                          C. 21                                          D. 27
13. 甲乙两人参加校内“信奥”知识竞赛，其中有 6 道选择题和 4 道判断题，甲乙两人依次各抽 1 题。请问甲乙两人中至少有一人抽到选择题的概率是 ( )。
- A. 4/15                                      B. 13/15                                      C. 4/5                                          D. 11/15

14. 如下图所示，用 4 种颜色给图中的 A、B、C、D、E、F 这 6 个点涂色，要求每个点涂一种颜色，且图中每条线段的两个端点涂不同颜色，则不同的涂色方法有 ( ) 种。



- A. 288                                      B. 264                                      C. 240                                      D. 168

15. 1800 的所有约数的和为 ( )。
- A. 6043                                      B. 6044                                      C. 6045                                      D. 6046

二、阅读程序（程序输入不超过数组或字符串定义的范围；判断题正确填√，错误填×；除特殊说明外，判断题每题 1.5 分，选择题每题 3 分，共计 40 分）

(1)

```
01 #include <iostream>
02 using namespace std;
03
04 const int N = 100500;
05 const int Mod = 1000000007;
06 int n, m, j, p[N], f[N];
07 char s[N], t[N];
08
09 int main() {
10 scanf("%s", s+1); scanf("%s", t+1);
11 n = strlen(s+1); m = strlen(t+1);
12 j = 0; p[1] = 0;
13 for (int i = 1; i < m; ++i) {
14 while (j > 0 && t[j+1] != t[i+1]) j = p[j];
15 if (t[j+1] == t[i+1]) ++j;
16 p[i+1] = j;
17 }
18 j = 0; f[0] = 1;
19 for (int i = 1; i <= n; ++i) {
20 f[i] = f[i-1];
21 while (j > 0 && s[i] != t[j+1]) j = p[j];
22 if (s[i] == t[j+1]) ++j;
23 if (j == m) {
24 j = p[j];
25 (f[i] += f[i-m]) %= Mod;
26 }
27 }
28 printf("%d\n", f[n]);
29 return 0;
30 }
```

■ 判断题

16. 若输入数据导致  $m > n$ ，则输出一定为 1。 ( )
17. 若输入数据导致  $m=1$ ，则输出一定为 2 的整次幂。 ( )

18. 去掉第 12 行, 不影响程序的输出结果。 ( )
19. 若将本程序改为可读入多组数据, 则并不用增加新的初始化代码。 ( )

### ■ 选择题

20. 若输入为 "xxxxxx xx", 则输出为 ( )。
- A. 5                      B. 8                      C. 10                      D. 15
21. 输入数据的两个字符串长度分别为  $N$ 、 $M$ , 则本程序的时间复杂度为 ( )。
- A.  $O(M)$                       B.  $O(N)$                       C.  $O(M+N)$                       D.  $O(MN)$

(2)

```
01 #include <iostream>
02
03 const int mod=1e4;
04 int n;
05 int ans;
06
07 inline int qpow(int a, int b) {
08 int res = 1;
09 for (; b; b >>= 1, a = 1LL * a * a % mod)
10 if (b & 1) res = 1LL * res * a % mod;
11 return res;
12 }
13
14 inline void add(int & x, const int & y) {
15 x += y;
16 if (x >= mod) x -= mod;
17 }
18
19 inline int calc(int n) {
20 int res = 0;
21 int c = 1, x = 1;
22 for (; x * 10 - 1 <= n; ++c, x *= 10)
23 add(res, 9LL * x * c % mod);
24 add(res, 1LL * c * (n-x+1) % mod);
25 return res;
26 }
27
```

```

28 int main() {
29 std::cin >> n;
30 if (n == 1) return puts("4"), 0;
31 ans = n % mod;
32 add(ans, 1ll * qpow(2, n-1) * n % mod);
33 add(ans, (calc(n) - 1 + mod) % mod);
34 add(ans, 1ll * qpow(2, n-1) * calc(n) % mod);
35 add(ans, (qpow(2, n) - 1 + mod) % mod);
36 add(ans, 2 * (n-1) % mod);
37 std::cout << ans << std::endl;
38 return 0;
39 }

```

### ■ 判断题

22. 第 14 行的两个 "g" 符号的作用一致。 ( )
23. 第 16 行等价于把 x 对 mod 取模，只是更慢。 ( )
24. 输入为 5 的时候，输出为 209。 ( )
25. 输入每增加 1，输出扩大 2 倍以上。 ( )

### ■ 选择题

26. 输入为 0 的时候，程序会 ( )。
- A. 输出乱码      B. 死循环      C. 输出 "0"      D. 输出 "-1"
27. 本程序的时间复杂度为 ( )。
- A.  $O(\log N)$       B.  $O(\log \log N)$       C.  $O(N)$       D.  $O(\log_2 N + \log N)$

(3)

```

01 #include <iostream>
02 using namespace std;
03 typedef long long i64;
04
05 const int NN = 1e7+5;
06 i64 ans, l, r;
07 bool s[NN];
08 int p[NN], o[NN], cnt;
09
10 inline void sss() {

```

```

11 o[1] = 1;
12 for (int i = 2; i <= r; ++i) {
13 if (is[i])
14 o[p[++cnt] = i] = i;
15 for (int j = 1; j <= cnt && p[j] * i <= r; ++j) {
16 s[p[j] * i] = true;
17 o[p[j] * i] = p[j];
18 if (i % p[j] == 0) break;
19 }
20 }
21 }
22
23 inline void answer() {
24 sss();
25 for (int i = 1; i <= r; ++i)
26 if (s[i]) ans += i/o[i];
27 cout << ans << endl;
28 }
29
30 int main() {
31 cin >> l >> r;
32 answer();
33 }

```

### ■ 判断题

28. (2分) 本题使用了埃氏筛法。 ( )
29. (2分) 如果输入数据  $l$  比  $r$  大, 那么程序会报错或者发生死循环。 ( )
30. (2分) 本程序可以处理的最大  $r$  是 10000000。 ( )
31. (2分) 在确保  $l < r$  的前提下,  $l$  的大小不影响程序的运行速度。 ( )

### ■ 选择题

32. (4分) 当输入为 "6 27" 时, 输出为 ( )。
- A. 115      B. 116      C. 117      D. 118
33. (4分) 本程序的时间复杂度为 ( )。
- A.  $O(\log N)$       B.  $O(N)$       C.  $O(\log \log N)$       D.  $O(N \log \log N)$

### 三、完善程序（单选题，每小题 3 分，共计 30 分）

(1) 输入一系列短棍的长度，它们是由一些等长的木棒截断得来的。请输出原始木棒的最小可能长度。截断后的短棍不超过 64 根，且长度不超过 50。本程序使用搜索+剪枝的办法来解决该问题，共使用 4 个剪枝。

比如，输入数据为：

9

5 2 1 5 2 1 5 2 1

输出数据为：

6

拼凑方案为：

5+1, 5+1, 5+1, 2+2+2

```
01 #include <iostream>
02 using namespace std;
03
04 int a[100], v[100], n, len, cnt;
05 bool dfs(int stick, int cab, int last) {
06 if (stick > cnt) return true;
07 if (cab == len) return dfs(stick + 1, 0, 1);
08 int fail = 0;
09 for (①)
10 if (!v[i] && cab + a[i] <= len && fail != a[i]) {
11 v[i] = 1;
12 if (dfs(stick, cab + a[i], i+1)) return true;
13 ②;
14 v[i] = 0;
15 if (③||④) return false;
16 }
17 return false;
18 }
19
20 int main() {
21 while (cin >> n && n) {
22 int sum = 0, val = 0;
23 for (int i = 1; i <= n; i++) {
24 scanf("%d", &a[i]);
25 sum += a[i]; val = max(val, a[i]);
```

```
26 }
27 sort(a + 1, a + n + 1);
28 reverse(a + 1, a + n + 1);
29 for (len = val; len <= sum; len++) {
30 if (sum % len) continue;
31 cnt = sum / len;
32 memset(v, 0, sizeof(v));
33 if ((5)) break;
34 }
35 cout << len << endl;
36 }
37 return 0;
38 }
```

34. ①处应填 (     )。

- A. int i = n; i >= last; i--
- B. int i = last; i <= n; i++
- C. int i = last + 1; i <= n; i++
- D. int i = n; i >= last + 1; i--

35. ②处应填 (     )。

- A. fail += a[i]
- B. a[i] = fail
- C. fail = cab+a[i]
- D. fail = a[i]

36. ③处应填 (     )。

- A. cab == 0
- B. cab == a[i]
- C. a[last] == fail
- D. last == 0

37. ④处应填 (     )。

- A. cab + a[i] <= len
- B. a[i] == fail
- C. cab + a[i] == len
- D. a[i] > cab / 2

38. ⑤处应填 (     )。

- A. dfs(0, 0, 1)
- B. dfs(1, 0, 1)
- C. dfs(0, 0, 0)
- D. dfs(1, 0, 0)

- (2) 给定一个长度不超过 17 的字符串及整数  $K$ , 字符串由 0~9 组成, 且无前导 0。求使用其中的数字排列成无前导 0 且能被 17 整除的第  $K$  小的数。 $K \leq 17!$ 。

比如, 输入为 22422232, 输出为 2242232。

使用状态压缩 DP,  $f[n][val][State]$  表示从个位 (个位为第 1 位) 填到第  $n$  位时, 还没有用过的数字对应的状态编号为  $State$ , 还没有用过的数字构成的数对 17 取模的结果为  $val$ , 且整体为 17 倍数的情况下的合法方案数 (现在的  $n$  位数加上  $val \times 10^n$  之后是 17 的倍数)。

```
01 #include <iostream>
02 using namespace std;
03
04 typedef long long ll;
05 const int N=20, M=20005;
06 int cnt[N]; char s[N];
07 ll f[N][N][M], d[N], K;
08
09 inline ll dfs(int k, int val, int sta) {
10 ①;
11 if (~f[k][val][sta]) return f[k][val][sta];
12 ll res = 0;
13 for (int i = 0; i <= 9; ++i) {
14 int lim = sta/d[i] % (cnt[i] + 1);
15 if (lim == cnt[i]) continue;
16 res += ②;
17 }
18 return f[k][val][sta] = res;
19 }
20
21 int main() {
22 ③;
23 scanf("%s", s+1); cin >> K;
24 int len = strlen(s+1);
25 for (int i = 1; i <= len; ++i)
26 ++cnt[s[i]-'0'];
27 d[0] = 1;
28 for (int i = 1; i <= 9; ++i)
29 d[i] = ④;
30 int sta = 0, val = 0; ll now = 0;
31 for (int i = len; i >= 1; --i)
32 for (int j = i == len?1:0; j <= 9; ++j) {
```

```
33 int lim = ⑤;
34 if (lim == cnt[j]) continue;
35 ll tmp = dfs(i-1, (val*10+j)%17, sta+d[j]);
36 if (now + tmp >= K) {
37 putchar(j+'0');
38 val = (val * 10 + j) % 17;
39 sta += d[j];
40 break;
41 }
42 now += tmp;
43 }
44 cout << endl;
45 return 0;
46 }
```

39. ①处应填 (     )。

- A. if (k) return !val                      B. if (!k) return !val  
C. if (!k) return val                      D. if (k) return val

40. ②处应填 (     )。

- A. dfs(k+1, (val - i \* 10) % 17, sta + d[i])  
B. dfs(k-1, (val \* 10 + i) % 17, sta - d[i])  
C. dfs(k+1, (val - i \* 10) % 17, sta - d[i])  
D. dfs(k-1, (val \* 10 + i) % 17, sta + d[i])

41. ③处应填 (     )。

- A. memset(f,255,sizeof(f))                      B. memset(f,0,sizeof(f))  
C. memset(f,127,sizeof(f))                      D. memset(f,128,sizeof(f))

42. ④处应填 (     )。

- A. d[i-1] \* (cnt[i-1])                      B. d[i-1] \* (cnt[i])  
C. d[i-1] \* (cnt[i-1]+1)                      D. d[i-1] \* (cnt[i]+1)

43. ⑤处应填 (     )。

- A. sta / d[j] % (cnt[j]+1)                      B. sta / d[j-1] % (cnt[j])  
C. sta / d[j-1] % (cnt[j]+1)                      D. sta / d[j] % (cnt[j])



扫码获取  
答案解析

# 提高组 CSP-S 2024 初赛模拟卷 5

一、单项选择题（共 15 题，每题 2 分，共计 30 分；每题有且仅有一个正确选项）

1. 以下哪个命令会开启编译器几乎所有常用的警告？（     ）

- A. `g++ -g -lm main.cpp -o main`
- B. `g++ -g -Wall main.cpp -o main`
- C. `g++ -g -O2 main.cpp -o main`
- D. `g++ -g -std=c++11 main.cpp -o main`

2. 以下关于 NOI Linux 的文件操作描述错误的是（     ）。

- A. `mkdir` 是新建一个文件
- B. `pwd` 显示当前工作文件路径
- C. `ls` 显示文件及文件夹
- D. `more` 在终端中分页显示普通文本类型文件

3. 以下哪个不属于 STL 中算法模板库的操作函数？（     ）

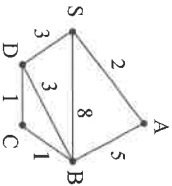
- A. `reverse`            B. `sort`            C. `push_back`        D. `lower_bound`

4. 以下哪个说法是不正确的？（     ）

- A. `tuple` 中的每个值都必须相同的类型
- B. `pair` 定义在头文件 `utility` 中，用于将两个值组合在一起存储
- C. `multiset` 是维护有序可重集合的容器
- D. `set` 是维护有序不可重集合的容器

5. 对下图使用 Floyd 算法计算 S 点到其余各点的最短路径长度时，到 B 点的距离  $d[B]$  初始时赋为 8，在算法的执行过程中不可能出现的值是（     ）。

- A. 4                    B. 5                    C. 6                    D. 7



6. 平面上有 5 个点  $A(5, 3)$ ,  $B(3, 5)$ ,  $C(2, 1)$ ,  $D(3, 3)$ ,  $E(5, 1)$ 。以这 5 个点作为完全图  $G$  的顶点, 每两点之间的直线距离是图  $G$  中对应边的权值。图  $G$  的最小生成树中的所有边的权值和为 ( )。

A. 8                      B.  $7 + \sqrt{5}$                       C. 9                      D.  $6 + \sqrt{5}$

7. 单源最短路的 Dijkstra 算法写的 C++ 程序的运行时间复杂度是 ( )。

A.  $O(n^2 \log n)$                       B.  $O(n^3)$                       C.  $O(n^2)$                       D.  $O(n \log n)$

8. 对于一棵二叉树, 独立集是指两两互不相邻的结点构成的集合。比如, 图 1 有 5 个不同的独立集 (1 个双点集合、3 个单点集合、1 个空集), 图 2 有 14 个不同的独立集。那么, 图 3 有 ( ) 个不同的独立集。

A. 1936                      B. 3600                      C. 5536                      D. 6300

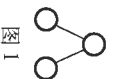


图 1

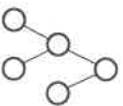


图 2

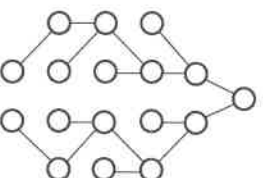


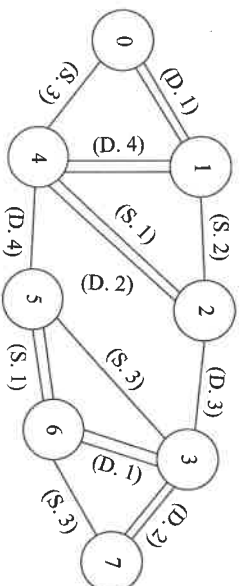
图 3

9. 以下哪个说法是正确的? ( )

A. 要判断图是否连通, 只能用广度优先搜索算法来判别  
B. 高斯是图论的创始人之一, 他解决了哥尼斯堡七桥问题  
C. 图灵的主要贡献是用图灵机破解了德军的 Enigma 密码  
D. 如果待查找的元素确定, 只要哈希表的大小不小于查找元素的个数, 就一定不会产生冲突的哈希函数

10. 下图中的 8 个结点通过 13 条无向边相连, 每条无向边上都有两项信息: 前面的字母表示这条边的安全性 (字母 S 表示这条边是 Safe 边, 字母 D 表示这条边是 Danger 边), 后面的数字表示这条边的长度。从中找出 7 条边能连通这 8 个结点, 且这 7 条边中恰好有 2 条 Safe 边 (S 边), 记这 7 条边长度之和为 path, 则 path 的最小值为 ( )。

A. 10                      B. 11                      C. 12                      D. 13



11. 以下哪个不是 C++ 语言中面向对象模式的主要性质? ( )
- A. 封装性      B. 继承性      C. 单调性      D. 多态性
12. 从 1 到 2021 的所有奇数中, 至少要选出 ( ) 个数, 才能确保其中必定存在两个数, 它们的和是 2022?
- A. 505      B. 506      C. 507      D. 508

13. 如果一棵二叉树有 2024 个结点, 树中的结点按层次和顺序依次编号(每层中从左到右编号), 其中根结点为 1, 那么编号为 1025 的父结点为第 ( ) 层第 ( ) 个结点。
- A. 10   1      B. 10   2      C. 9   256      D. 11   2

14. 4 个学生和 2 个老师围绕圆桌入座, 2 个老师之间至少有 1 个学生, 有多少种就座方法? ( )
- A. 36      B. 72      C. 48      D. 84

15. 由数字 1, 2, 3, 4, 5, 6, 7 组成无重复数字的 7 位整数, 从中任取一个, 所取的数满足首位是 1, 且任意相邻两位数字之差的绝对值不大于 2, 则取到此类数的概率为 ( )。
- A. 1/120      B. 1/240      C. 1/360      D. 1/480

二、阅读程序 (程序输入不超过数组或字符串定义的范围; 判断题正确填√, 错误填×; 除特殊说明外, 判断题每题 1.5 分, 选择题每题 3 分, 共计 40 分)

(1)

```
01 #include <iostream>
02 #include <algorithm>
03 #include <string.h>
04
05 using namespace std;
```

```
06
07 const int N = 100000, M = 100000;
08 const long long mod = 1e8-3;
09
10 struct StMatch{
11 int a, b;
12 } matches[N+1];
13 bool operator < (const StMatch a, const StMatch b) {
14 return a.a < b.a;
15 }
16
17 struct StTreeArray {
18 int tree[M+1];
19 int lb(int x) { return x & (-x); }
20 void add(int x, int t) {
21 while (x <= M) {
22 tree[x] += t;
23 x += lb(x);
24 }
25 }
26 int sum(int x) {
27 int ans = 0;
28 while (x > 0) {
29 ans += tree[x];
30 x -= lb(x);
31 }
32 return ans;
33 }
34 };
35
36 StTreeArray t;
37 int n;
38 int a[N+2], b[N+2], pos[N+2];
39
40 int index(int a[], int x) {
41 int l = 1, r = n;
42 while (l < r) {
43 int m = (l + r) >> 1;
44 if (a[m] == x) return m;
45 if (a[m] < x) {
```

```
46 l = m + 1;
47 } else {
48 r = m - 1;
49 }
50 }
51 if (a[l] == x) return l;
52 if (a[r] == x) return r;
53 return 0;
54 }
55
56 int main() {
57 cin >> n;
58 for (int i = 1; i <= n; i++) {
59 cin >> matches[i].a;
60 a[i] = matches[i].a;
61 }
62 for (int i = 1; i <= n; i++) {
63 cin >> matches[i].b;
64 b[i] = matches[i].b;
65 }
66 sort(a+1, a+n+1);
67 sort(b+1, b+n+1);
68 for (int i = 1; i <= n; i++) {
69 matches[i].a = index(a, matches[i].a);
70 matches[i].b = index(b, matches[i].b);
71 pos[matches[i].b] = i;
72 }
73 for (int i = 1; i <= n; i++) {
74 matches[i].a = pos[matches[i].a];
75 }
76 long long ans = 0;
77 for (int i = n; i >= 1; i--) {
78 ans += t.sum(matches[i].a - 1);
79 ans %= mod;
80 t.add(matches[i].a, 1);
81 }
82 cout << ans << endl;
83 return 0;
84 }
```

## ■ 判断题

16. 第 19 行就是计算 `lowbit`，即返回 `x` 表示成二进制后最右边的 1 在哪一位。 ( )
17. 将第 13、14 行中的 “<” 同时或者其中之一改成 “>”，不影响程序结果。 ( )
18. 第 52 行可以去掉，不影响程序结果。 ( )
19. 第 69 行也可以去掉，不影响程序结果。 ( )

## ■ 选择题

20. 本程序的时间复杂度是 ( )。

A.  $O(n^2 \log n)$       B.  $O(\log n)$       C.  $O(n)$       D.  $O(n \log n)$

21. 对于输入 “4 1 3 4 2 1 7 2 4”，本程序的输出为 ( )。

A. 1      B. 2      C. 3      D. 4

(2)

```
01 #include<stdio>
02 using namespace std;
03 int gcd(int a,int b) {
04 return b==0?a:gcd(b,a%b);
05 }
06 int main() {
07 int T;
08 scanf("%d",&T);
09 while(T--) {
10 int a0,a1,b0,b1;
11 scanf("%d%d%d%d",&a0,&a1,&b0,&b1);
12 int p=a0/a1,q=b1/b0,ans=0;
13 for(int x=1;x*x<=b1;x++)
14 if(b1%x==0){
15 if(x%a1==0&&gcd(x/a1,p)==1&&gcd(q,b1/x)==1) ans++;
16 int y=b1/x;
17 if(x==y) continue;
18 if(y%a1==0&&gcd(y/a1,p)==1&&gcd(q,b1/y)==1) ans++;
19 }
20 printf("%d\n",ans);
21 }
22 return 0;
23 }
```

## ■ 判断题

22. 将第 17 行中的 `continue` 改为 `break`, 不影响程序结果。 ( )
23. 如果输入是 "1 41 1 96 288", 则输出是 "5"。 ( )
24. 第 13 行中的 `x=1` 可以换为 `x=a1`, 程序的运行速度一定会更快。 ( )
25. 本题有复杂度更优的做法。 ( )

## ■ 选择题

26. 本题实质上是求符合下面哪个条件的 `x` 的个数? ( )
- A. `gcd(a0,x)=a1` 且 `lcm(x,b1)=b0`
- B. `lcm(a1,x)=a0` 且 `lcm(x,b0)=b1`
- C. `gcd(a0,x)=a1` 且 `gcd(x,b1)=b0`
- D. `gcd(a0,x)=a1` 且 `lcm(x,b0)=b1`
27. 如果输入是 "1 95 1 37 1776", 则输出是 ( )。
- A. 1                      B. 2                      C. 3                      D. 4

(3)

```
01 #include <iostream>
02 #include <string.h>
03 #include <math.h>
04 #include <algorithm>
05 using namespace std;
06
07 const int N = 10010, MONEY = 1010, L = 1010;
08
09 struct sline{
10 int st, en, fun, cost;
11 };
12 int l, n, money;
13 sline line[N];
14 int f[L][MONEY], use[L][MONEY], jump[L][MONEY];
15 void print(int l, int cost);
16 bool comp(sline & la, sline & lb) {
17 return (la.en < lb.en ||
18 (la.en == lb.en && la.st < lb.st));
19 }
20
```

```
21 int main() {
22 cin >> l >> n >> money;
23 for (int i = 0; i < n; i++) {
24 cin >> line[i].st >> line[i].en
25 >> line[i].fun >> line[i].cost;
26 line[i].en += line[i].st;
27 }
28 sort(line, line + n, comp);
29
30 memset(f, 128, sizeof(f));
31 f[0][0] = 0; jump[0][0] = -1; use[0][0] = -1;
32 for (int i = 0; i < n; i++) {
33 for (int c = money; c >= line[i].cost; c--) {
34 int st = line[i].st;
35 if(f[st][c - line[i].cost] + line[i].fun >
36 f[line[i].en][c]) {
37 jump[line[i].en][c] = st;
38 use[line[i].en][c] = i;
39 }
40 f[line[i].en][c] = max(f[line[i].en][c],
41 f[st][c - line[i].cost] + line[i].fun);
42 }
43 }
44 int ans = -1, sol = 0;
45 for (int c = 0; c <= money; c++) {
46 if (f[l][c] > ans) {
47 sol = c;
48 }
49 ans = max(ans, f[l][c]);
50 }
51 if (ans >= 0) {
52 cout << ans << endl;
53 cout << "sol = " << sol << endl;
54 print(l, sol);
55 }
56 else cout << -1 << endl;
57 return 0;
58 }
59 void print(int l, int cost) {
60 if (l <= 0) return ;
61 print(jump[l][cost], cost - line[use[l][cost]].cost);
```

```
62 cout << "line : " << use[l][cost]
63 << " st : " << line[use[l][cost]].st
64 << " en : " << line[use[l][cost]].en
65 << " fun : " << line[use[l][cost]].fun
66 << " cost : " << line[use[l][cost]].cost << endl;
67 return ;
68 }
```

### ■ 判断题

28. (2 分) 第 16~19 行是按照线段的终点排序, 且终点越大的越靠前, 终点相同则看起点。 ( )
29. (2 分) 第 30 行中的 128 可以换成别的数。 ( )
30. (2 分) 第 33 行倒序进行动态规划, 是为了节约空间, 使得 f 数组可以少一维。 ( )
31. (2 分) 本题涉及线段的代价和收益, 其实也可以使用最小费用最大流来解决。 ( )

### ■ 选择题

32. (4 分) 若  $N$ 、 $M$ 、 $L$  分别为最大线段数、最大代价限制、最长区间长度, 则本程序的时间复杂度为 ( )。
- A.  $O(NL)$       B.  $O(NML)$       C.  $O(ML)$       D.  $O(MN)$
33. (4 分) 如果使用最小费用最大流来解决本题, 则和本程序相比, 网络流解法的时间复杂度 ( )。
- A. 不能使用网络流来解决      B. 更大
- C. 更小      D. 无法确定

### 三、完善程序 (单选题, 每小题 3 分, 共计 30 分)

- (1) 给定数  $a$  和  $b$ , 对  $a$  可以进行 3 种操作:  $\times 2$ ,  $/2$ ,  $+1$ 。问: 最少要进行多少次操作可以让  $a$  变为  $b$ ? 比如, 输入数据为 31 13, 输出为 8, 最佳操作顺序为:  $31 \rightarrow 32 \rightarrow 16 \rightarrow 8 \rightarrow 9 \rightarrow 10 \rightarrow 11 \rightarrow 12 \rightarrow 13$ 。

本题不能简单地进行搜索, 因为前两个操作正好相互抵消, 可以无限搜索下去。本程序首先使  $a$ 、 $b$  满足  $a < b < 2a$ , 然后尝试如何从  $a$  变成  $b$ 。

```
01 #include<bits/stdc++.h>
02 #define int long long
03 using namespace std;
04 signed main() {
05 int a,b;
06 scanf("%lld%lld",&a,&b);
07 int cnt = 0;
08 while(a > b) {
09 cnt++;
10 if(a % 2 == 1) ①;
11 else a /= 2;
12 }
13 while(b >= a * 2) {
14 cnt++;
15 if(b % 2 == 1) ②;
16 else b /= 2;
17 }
18 if(③) {
19 printf("%lld\n",cnt);
20 return 0;
21 }
22 int aa = a,bb = b;
23 int minn=0x3f3f3f3f, bs=0;
24 while(aa>1 && bb>=1 && aa<=bb) {
25 minn=min(minn, cnt+bs+(bb-aa));
26 if(aa % 2 == 1) ④;
27 aa /= 2;
28 bs++;
29 if(bb % 2==1) ⑤;
30 bb /= 2;
31 bs++;
32 }
33 printf("%lld\n",minn);
34 return 0;
35 }
```

34. ①处应填 (     )。

A. a++

B. a--

C. a\*=2

D. a/=2

35. ②处应填 ( )。

- A. b++                      B. b--                      C. b\*=2                      D. b/=2

36. ③处应填 ( )。

- A. a = b                      B. a == b-1                      C. a = b-1                      D. a == b

37. ④处应填 ( )。

- A. aa--,bs--                      B. aa--,bs++                      C. aa++,bs--                      D. aa++,bs++

38. ⑤处应填 ( )。

- A. bb--,bs--                      B. bb++,bs++                      C. bb--,bs++                      D. bb++,bs--

(2) 输入一个字符串 s，求其删掉若干个字符之后，最多能出现多少个“jessie”，并且，如果给出删掉每个字符的代价，则需求出在保证代价最小的情况下“jessie”最多出现多少次。

比如，输入数据为：

```
jessie
1 1 5 4 6 1 1
```

输出为：

```
1
4
```

(因为 3 个 s 的代价分别为 5、4、6，所以删掉的是第二个 s)

又比如，输入数据为：

```
jejesconsiete
6 5 2 3 6 5 7 9 8 1 4 5 1

输出为：
1
21
```

```
01 #include <cstdio>
02 #include <cstring>
03 #include <iomanip>
04 #include <iostream>
05 using namespace std;
06 const int maxn = 300005;
```

```
07 char s[maxn];
08 int c[maxn][8], t[maxn][8], p[8][maxn], d[maxn];
09 int dp[maxn][8], sum[maxn], ans[maxn];
10 string jes = "jessie";
11 int main() {
12 scanf("%s", s + 1);
13 int n = strlen(s + 1);
14 for (int i = 1; i <= n; i++) {
15 cin >> d[i];
16 sum[i] = sum[i - 1] + d[i];
17 }
18 ①;
19 for (int i = 1; i <= n; i++) {
20 memcpy(dp[i], dp[i - 1], sizeof(int) * 8);
21 if (s[i] == 'j') {
22 dp[i][1] = max(dp[i][1], dp[i][0]);
23 } else if (s[i] == 'e') {
24 dp[i][2] = max(dp[i][2], dp[i][1]);
25 dp[i][6] = max(dp[i][6], dp[i][5]);
26 ②;
27 } else if (s[i] == 's') {
28 dp[i][4] = max(dp[i][4], dp[i][3]);
29 dp[i][3] = max(dp[i][3], dp[i][2]);
30 } else if (s[i] == 'i') {
31 dp[i][5] = max(dp[i][5], dp[i][4]);
32 }
33 }
34 cout << dp[n][6] << endl;
35 memset(p, 0x7f, sizeof(p));
36 memset(t, 0x7f, sizeof(t));
37 memset(c, 0x7f, sizeof(c));
38 memset(ans, ③, sizeof(ans));
39 t[0][0] = sum[n];
40 p[0][1] = sum[n];
41 ans[0] = 0;
42 for (int i = 1; i <= n; i++) {
43 p[0][1] = sum[n] - sum[i - 1];
44 for (int x = 6; x >= 1; x--) {
45 if (jes[x] == s[i]) {
46 if (x == 1)
47 c[i][x] = ans[dp[i][x] - 1];
48 else
```

```
49 c[i][x] = ④;
50 t[i][x] = c[i][x] + sum[n] - sum[i];
51 p[x][dp[i][x]] = min(p[x][dp[i][x]], t[i][x]);
52 if (x == 6)
53 ans[dp[i][x]] = ⑤;
54 }
55 }
56 }
57 cout << ans[dp[n][6]];
58 return 0;
59 }
```

39. ①处应填 ( )。

- A. `dp[0][0] = 0`                      B. `dp[0][0] = 1`  
C. `dp[0][0] = -1`                     D. `dp[0][0] = 6`

40. ②处应填 ( )。

- A. `dp[i][0] = dp[i][6]`  
B. `dp[i][0] = dp[i][6] - 1`  
C. `dp[i][0] = dp[i][6] + 1`  
D. `dp[i][0] = max(dp[i][0], dp[i][6] + 1)`

41. ③处应填 ( )。

- A. `0x00`                      B. `0xff`                      C. `0x8f`                      D. `0x4f`

42. ④处应填 ( )。

- A. `p[x - 1][dp[i][x]] - sum[n] + sum[i - 1]`  
B. `p[x - 1][dp[i][x]] - sum[n] + sum[i]`  
C. `p[x - 1][dp[i][x]] - sum[n - 1] + sum[i]`  
D. `p[x - 1][dp[i - 1][x]] - sum[n] + sum[i - 1]`

43. ⑤处应填 ( )。

- A. `min(ans[dp[i][x]], t[i][x])`  
B. `min(ans[dp[i][x]], p[i][x])`  
C. `min(ans[dp[i][x]], d[i][x])`  
D. `min(ans[dp[i][x]], c[i][x])`

提高组 CSP-S 2024 初赛模拟卷 6



取析 获解 码案 扫描

一、单项选择题（共15题，每题2分，共计30分；每题有且仅有一个正确选项）

1. 在 Linux 系统终端中，cp 命令的作用是( )。

- A. 切换工作目录
- B. 远程复制文件
- C. 复制文件或目录
- D. 在文件与文件之间建立 couple 关系

2. 8位无符号二进制数10110001和10110111相加,运算结果为无符号二进制数( )。

- A. 101101000      B. 01101000  
C. 101001000      D. 01001000

3. 给出一个数字序列 1 5 2 7 3 6 4, 现希望通过每次交换其中两个数字, 使得序列变成数字序列 5 7 4 2 6 3 1, 那么最少需要交换多少次? ( )

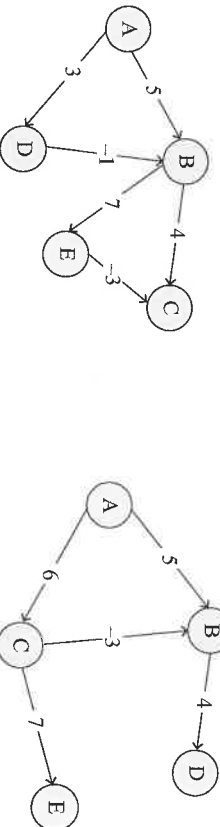
- A.3      B.4      C.5      D.6

4. 快速排序是一种关键字比较排序算法，需要比较和交换这两个环节。若选择的比较基准值总是当前子序列的第一个数字，那么对于数字序列 1 2 3 4 5 6 7 8，使用快速排序让其从小到大排序将需要多少次交换？（ ）

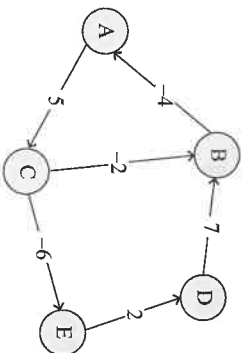
- A.4      B.36      C.72      D.0

5. 在求解最短路径问题的算法中，Dijkstra 算法在处理没有负权边的图时才能保证正确。但在有些出现了负权边的图中，该算法也能得到正确答案。下面的选项起始点都为点 A，请问哪张图使用 Dijkstra 算法可以正确地求出起始点到所有其他点的最短距离？（ ）

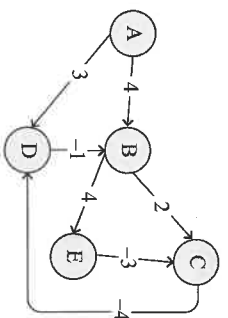
- A.



C.



D.



6. 现在有一棵树用 DFS 遍历得到的点为 ABCD, 用 BFS 遍历得到的点也为 ABCD, 同一层的结点从左到右顺序遍历。请问在这棵树有可能的形态上得到的平均深度是多少 (不同的形态为至少有两个不同的点在不同深度, 根结点的深度为 1)? ( )

A. 1.5

B. 3.0

C. 2.0

D. 1.0

7. 现给出一个深度为  $n$  ( $n \geq 2$ ) 的完美二叉树, 请问自下往上、自右往左删掉多少点, 一定可以得到深度为  $n-1$  的完全二叉树? ( )

A.  $2^{n-1} + 2^{n-2}$ B.  $2^{n-2}$ C.  $2n$ D.  $2^{n-1}$ 

8. 给出数字序列 1 2 3 4 5 6, 对这些数字进行排列, 假设分别用  $x_1 \sim x_6$  表示排列后的数字。对于每个数字  $x_i$  保证  $x_{i+1}$ 、 $x_{i+2} \leq x_i + 2$ 。请问这样的排列方式有多少种? ( )

A. 50

B. 60

C. 70

D. 80

9. 食堂有 8 个档口, 编号为 1~8, 每个档口有一个美味值。同学们常常会因为不知道吃什么而摇摆不定。小豪想出了一种非常棒的选择方法。现在有 8 个档口可供选择, 使用骰子, 若摇出了 1~2, 则从 1~4 号档口中继续选择, 否则从 5~8 号档口中继续选择。以此类推, 直到只剩下一个档口。假设每个档口的美味值就是其编号, 请问小豪能吃到的美味值的期望与以下哪个值最接近? ( )

A. 5.7

B. 6.2

C. 5.0

D. 4.5

10. 倘若逆波兰表达式 (又称后缀表达式) 的值为 3, 该逆波兰表达式的部分表示是: 7 9 2 + ? 1 3 \* + 4 +。请问“?”处应该是什么符号? ( )

A. +

B. \*

C. /

D. -

11. 若某算法的运行时间满足  $T(n) = 128T\left(\frac{n}{2^{2023}}\right) + \sqrt[290]{n}$ , 则  $T(n) = ( )$ 。

A.  $O(n^{1/289})$ B.  $O(n^{1/290})$ C.  $O(n^{1/289} \lg n)$ 

D. 无法确定

12. 班级里有 4 位学生参加了知识竞赛。知识竞赛有 4 道判断题，4 位同学的答题情况若分别用 0 和 1 来表示选了错和对，那么答题情况如下：1100、1011、0011、1111。现已知这 4 人最多答对了两道题，请你找出一组答案。( )

A. 1100      B. 0111      C. 0000      D. 1111

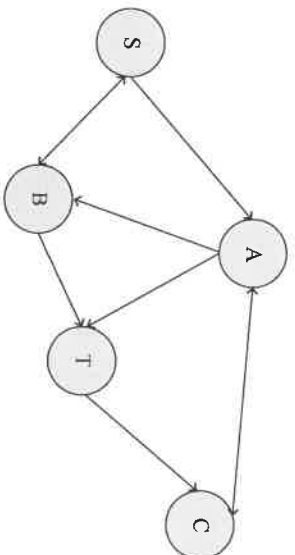
13. 母函数通过多项式来计算组合数学题目，比如有 1 克砝码、2 克砝码、3 克砝码，每种重量各有几种可能的获取方案？对于 1 克砝码，我们可以取或不取，得到的重量贡献分别是 1 和 0，对应的母函数是  $1 \times x^0 + 1 \times x^1 = 1 + x$ ，其中  $x^0$  表示不取砝码产生的贡献， $x^1$  表示取砝码产生的贡献。对于 2 克砝码，对应的母函数是  $1 \times x^0 + 1 \times x^2 = 1 + x^2$ 。对于 3 克砝码，对应的母函数是  $1 \times x^0 + 1 \times x^3 = 1 + x^3$ 。将三者的母函数相乘， $(1+x)(1+x^2)(1+x^3) = 1 + x + x^2 + 2x^3 + x^4 + x^5 + x^6$ ，所以可得到 7 种重量。再作深一些的解析，三者母函数产生的  $2x^3$ ，代表三者可组成两种不同情况总重量为 3 的砝码。 $x^3$  可以由  $1 \times 1 \times x^3$  得到，代表 1 克砝码、2 克砝码不取，3 克砝码取；也可以由  $x \times x^2 \times 1$  得到，代表 1 克砝码、2 克砝码取，3 克砝码不取。问：各位数字之和等于 11 的三位数个数是 ( )。

A. 59      B. 62      C. 60      D. 61

14. 在一个三维坐标系中有两个正方体。现已知两个正方体的所有顶点的坐标，请问最少需要多少时间复杂度能够判断两个正方体是否相交？( )

A.  $O(n^2)$       B.  $O(1)$       C.  $O(n \log n)$       D. 无法判断

15. 对于下图，假设起点为 S，终点是 T，从 S 到 T 的所有路径（包含复杂路径）的数量是 ( )。



A. 1      B. 2      C. 3      D. 4

二、阅读程序（程序输入不超过数组或字符串定义的范围；判断题正确填√，错误填×；除特殊说明外，判断题每题 1.5 分，选择题每题 3 分，共计 40 分）

(1)

```
01 #include<iostream>
02 #include<algorithm>
03 #include<cstring>
04 #include<queue>
05
06 using namespace std;
07
08 using pii = pair<int, int>;
09 using i64 = long long;
10 using uint = unsigned int;
11 using ui64 = unsigned long long;
12
13
14 const int N = 2e6 + 10, M = N * 2;
15
16 int a[N];
17 int st[N][25], logg[N];
18
19 int getrmx(int i, int mid){
20 int maxn = max(st[i][logg[mid - i + 1]], st[mid - (1 <<
 logg[mid - i + 1]) + 1][logg[mid - i + 1]]);
21 return maxn;
22 }
23
24 int q[N], sl[N], sr[N];
25
26 namespace IO {
27 char a; int f;
28 template<typename T> inline void read(T &x) {
29 x = 0; f = 1; a = getchar();
30 for(; ! isdigit(a); a = getchar()) if(a == '-') f = -1;
31 for(; isdigit(a); a = getchar()) x = x * 10 + a - '0';
32 x *= f;
33 }
34 template<typename T> inline void write(T x) {
35 if(x > 9) write(x / 10);
```

```
36 putchar(x % 10 + '0');
37 }
38 int read() { int x; read(x); return x; }
39 i64 read64() { i64 x; read(x); return x; }
40 }
41
42 using IO :: write;
43
44 int main(){
45 logg[1] = 0;
46 for(int i = 2; i <= 2e6; i++){
47 if((1 << (logg[i] - 1) + 1)) == i){
48 logg[i] = logg[i - 1] + 1;
49 jelse logg[i] = logg[i - 1];
50 }
51 memset(st,0,sizeof st);
52
53 int n;
54 cin >> n;
55 for(int i = 1; i <= n; i++) scanf("%d",&a[i]),st[1][0] = a[i];
56
57 for(int i = 1; i <= 24; i++){
58 for(int j = 1; j <= n; j++){
59 if(j + (1 << i - 1) <= n){
60 st[j][i] = max(st[j][i - 1],st[j +
61 (1 << i - 1)][i - 1]);
62 }
63 }
64
65 int l = 1,r = 0;
66 for(int i = 1; i <= n; i++){
67 while(r >= l && a[q[r]] > a[i]) r--;
68 if(r >= l) sl[i] = q[r];
69 else sl[i] = 0;
70 q[++r] = i;
71 }
72
73 l = 1,r = 0;
74 for(int i = n; i >= 1; i--){
75 while(r >= l && a[q[r]] > a[i]) r--;
76 if(r >= l) sr[i] = q[r];
```

```
77 else sr[i] = n + 1;
78 q[++r] = i;
79 }
80
81 __int128 ans = 0;
82 for(int i = 1; i <= n; i++){
83 __int128 k = sr[i] - 1 - sl[i] - 1 + 1;
84 ans = max(ans, k * getrmax(sl[i] + 1, sr[i] - 1) * a[i]);
85 }
86
87 write(ans);
88 return 0;
89 }
```

### ■ 判断题

16. 若将第 51 行去掉, 程序的输出不变。 ( )
17. 数组 `logg[i]` 求解的是以 2 为底 `i` 的对数。 ( )
18. 当 `n` 不为 1 时, 该程序 `ans` 输出的值不可能等于 `a[1]*a[1]*n`。 ( )
19. 当输入的数据为 5 3 1 2 4 5 时, `sr[3]` 的值为 5。 ( )

### ■ 选择题

20. 当输入为 9 5 11 15 17 19 3 19 9 4 时, 输出为 ( )。
- A. 855                      B. 836                      C. 570                      D. 513
21. 当输入 10 7 3 18 9 4 5 1 7 6 5 时, `sl[5]` 的输出为 ( )。
- A. 7                          B. 3                          C. 9                          D. 2

### (2)

```
01 #include<bits/stdc++.h>
02 using namespace std;
03
04 const int N = 1e5 + 10;
05
06 typedef long long ll;
07 ll a[N], f[N][20];
08
09 ll ks(ll a, ll b, ll p){
10 ll res = 0;
11 while(b){
```

```
12 if((b & 1) res = (res + a) % p;
13 a = (a + a) % p;
14 b >>= 1;
15 }
16 return res;
17 }
18
19 int main(){
20 ll p;
21 int n,m;
22 scanf("%d %d %lld", &n, &m, &p);
23 for(int i = 1; i <= n; i++){
24 scanf("%lld", &a[i]);
25 f[i][0] = a[i];
26 }
27
28 for(int i = 1; i <= log2(n) + 1; i++){
29 for(int j = 1; j <= n; j++){
30 if(j + (1 << (i - 1)) <= n){
31 f[j][i] = (f[j][i - 1] + f[j + (1 << (i - 1))]
32 [i - 1]) % p;
33 }
34 }
35
36 int l,r;
37 for(int i = 1; i <= m; i++){
38 scanf("%d %d", &l, &r);
39 int x = r - l + 1;
40 int x2 = x;
41 int d = 0; ll sum = 0;
42 while(x){
43 if((x & 1) sum += f[l][d], sum %= p, l += 1 << d;
44 d++;
45 x >>= 1;
46 }
47 printf("%lld\n", ks(sum, x2, p));
48 }
49 }
50 }
```

注：题目保证  $n, m \leq 10^5$ 。  $1 \leq a[l]$ ,  $p \leq 10^{18}$ 。

## ■ 判断题

22. 若去掉第 30 行, 并不会导致程序错误。 ( )
23. (3 分) 第 47 行输出的答案是  $\text{sum} \times 2\%$   $p(x)$ 。 ( )
24. 该算法可以用前缀和优化。 ( )

## ■ 选择题

25. 算法的时间复杂度为 ( )。
- A.  $O(n \log n)$  B.  $O(n \log n + \log p)$
- C.  $O(n \log n + m \log p)$  D.  $O((n+m) \log n + m \log p)$

## 26. 当题目输入

```
10 1 129323435551
1 2 3 4 5 6 7 8 9 10
4 9
```

时, 答案输出 ( )。

- A. 3518743761 B. 90224199 C. 234 D. 195

## 27. 该程序可以优化至的时间复杂度为 ( )。

- A.  $O((n+m) \log n)$  B.  $O(n+m)$
- C.  $O(n+m \log p)$  D. 无法优化

(3)

```
01 #include<bits/stdc++.h>
02 using namespace std;
03 typedef long long ll;
04 ll l,r,f[25][15][15],a[25];
05 ll dfs(ll pos,bool limit,bool lead,ll pre1,ll pre2){
06 if(!pos){
07 return 1;
08 }
09 ll ans = 0;
10 if(!limit && !lead && f[pos][pre1 + 1][pre2 + 1] != -1){
11 return f[pos][pre1 + 1][pre2 + 1];
12 }
13 ll up = limit ? a[pos] : 9;
14 for(ll i = 0; i <= up; i++){
```

```
15 if(i != pre1 && i != pre2){
16 ans += dfs(pos - 1, limit && i == up, lead && i,
17 (ilead || i) ? i : -1, pre1);
18 }
19 if(!limit && !lead){
20 f[pos][pre1 + 1][pre2 + 1] = ans;
21 }
22 return ans;
23 }
24
25 ll solve(ll x){
26 ll cnt = 0;
27 while(x){
28 a[++cnt] = x % 10;
29 x /= 10;
30 }
31 memset(f, -1, sizeof (f));
32 return dfs(cnt, 1, 1, -1, -1);
33 }
34 int main(){
35 cin >> l >> r;
36 cout << solve(r) - solve(l - 1);
37 return 0;
38 }
```

假设输入的  $l$  和  $r$  都是不超过  $10^{18}$  的自然数，本程序的目标是统计位于区间  $[l, r]$  中具有某种性质的正整数的数量。据此完成下列问题。

### ■ 判断题

28. 将第 7 行改为 `return 0;`，程序统计出的数量不会改变。 ( )
29. (3 分) 第 15 行的作用是保证对所有被统计到答案的数字中，不会出现连续的 3 个相等的正整数。 ( )
30. 当输入的  $l$  和  $r$  相等时，输出必定为 1。 ( )

### ■ 选择题

31. 若输入为 1 100，则输出为 ( )。
- A. 100                      B. 90                      C. 70                      D. 80

32. (4 分) 若输入为 1 1000, 则输出为 ( )。

- A. 991                      B. 688                      C. 738                      D. 832

### 三、完善程序 (单选题, 每小题 3 分, 共计 30 分)

(1) 数组  $a[N]$  共有  $N$  项, 是由  $M$  生成的:  $a[i]=M^i \bmod 1000000007$ , 其中第一项为  $a[0]$ , 不是  $a[1]$ , 最后一项是  $a[N-1]$ 。给定  $M, N, K$ , 请你求出数组  $a[N]$  中第  $K$  小的数。对于 100% 的数据,  $2 \leq M \leq 10\,000\,000$ ,  $1 \leq K \leq N \leq 50\,000\,000$ 。时限 1.5s, 空间限制为 131 072.0 KB。

为了减少空间的使用, 使用基数排序方法。尝试补全程序。

```
01 #include<iostream>
02 #include<cstring>
03 using namespace std;
04
05 const int N = 1e7 + 10, mod = 1e9 + 7;
06
07 int a[N], b[65540];
08
09 int main(){
10 int n, m, k;
11 cin >> m >> n >> k;
12
13 int g = ①;
14 int s = 1;
15 for(int i = 1; i <= n; i++){
16 b[②]++;
17 s = 1ll * s * m % mod;
18 }
19
20 int Hi = -1;
21 if(b[0] >= k) Hi = 0;
22 for(int i = 1; i <= 65535; i++){
23 b[i] += b[i - 1];
24 if(Hi == -1 && b[i] >= k) Hi = i, ③;
25 }
26
27 for(int i = 0; i <= 65535; i++) b[i] = 0;
28
```

```

29 s = 1;
30 for(int i = 1; i <= n; i++){
31 if(④) b[s & 65535]++;
32 s = 1ll * s * m % mod;
33 }
34
35 for(int i = 0; i <= 65535; i++){
36 if(k <= b[i]){
37 cout << ⑤ << endl;
38 break;
39 }
40 b[i + 1] += b[i];
41 }
42 return 0;
43 }

```

33. ①处应填 ( )。

A. 65534                      B. 65535 << 16

C. 5535 >> 16                D. 65536

34. ②处应填 ( )。

A. (s & g) >> 16      B. (s & g)

C. (s & g) << 16      D. (s >> 16) & g

35. ③处应填 ( )。

A.  $k -= b[i - 1]$       B.  $k += b[i - 1]$

C.  $k -= b[i]$       D.  $k += b[i]$

36. ④处应填 ( )。

A. (s & g) == hi

B. (s & g) >> 16 == hi

C. (s >> 16) & g == hi

D. (s & g) << 16 == hi

37. ⑤处应填 ( )。

A.  $\text{Hi} + i$                       B.  $\text{Hi} + i \ll 16$

C.  $(\text{Hi} + i) \ll 16$               D.  $\text{Hi} \ll 16 + i$

(2) 豪豪哥是一只猫，它总是喜欢打翻家中的物品。

豪豪哥刚来到家中时是一只力量为 1、精力有  $m$  点的小猫。在家中，一共有  $n$  个物品，其中编号为 1 的物品重量为 0，编号为 2 的物品重量为 1，以此类推。若豪豪哥的力量大于或等于物品的重量，它便可以将物品打翻并且自身力量增加 1 点，但是前提是豪豪哥有足够的精力去打翻这个物品，否则豪豪哥将累死（精力为 0 时并不会累死）。注意，每个物品都会消耗豪豪哥的精力。

现在豪豪哥想知道如果它总是尽力去打翻物品（要么累死，要么打翻所有物品），那么它的能力值的期望最终是多少。由于小数点比较麻烦，因此你只需要输出期望  $\times n!$  关于 1000000007 取模后的值就可以了。

输入格式：

第 1 行有两个数  $n, m$ ，表示有  $n$  个物品和豪豪哥的初始精力 ( $1 \leq n \leq 500\,000$ ,  $1 \leq m \leq 10^9$ )。接下来一行有  $n$  个数， $a_i$  表示编号为  $i$  的物品消耗的豪豪哥的精力 ( $0 \leq a_i \leq m$ )。

解题思路：

设  $f[i]$  为至少打翻  $i$  个物品的方案数，那么恰好打翻  $i$  个物品的方案数为  $f[i] - f[i+1]$ ，打翻物品的期望就为  $(f[1] - f[2] + 2 * (f[2] - f[3]) + \dots + n * (f[n] - f[n+1]))/n!$ ，答案是  $(f[1] - f[2] + 2 * (f[2] - f[3]) + \dots + n * (f[n] - f[n+1]))/n!$ ，

化简即为  $\sum_{i=1}^n f[i]$ 。

假设  $\text{sum}[i]$  为编号  $1 \sim i$  的物品消耗的精力的总和，利用  $\text{sum}[i]$  与  $m$  的比值求出  $f[i]$ 。

```
01 #include<iostream>
02 #include<cstring>
03 #include<queue>
04 #define x first
05 #define y second
06
07 using namespace std;
08
09 const int N = 5e5 + 10, mod = 1e9 + 7;
10 typedef pair<int, int> PII;
11 typedef long long ll;
12
13 ll a[N]; ll b[N], f[N];
14 ll fac[N];
15 priority_queue<PII, vector<PII>, ③ > q;
```

```
16
17 ll qpow(ll a,ll b){
18 if(b == -1) return 1;
19 ll s = 1;
20 while(b){
21 if(b & 1) s = s * a % mod;
22 a = a * a % mod;
23 b >>= 1;
24 }
25 return s;
26 }
27
28 int main(){
29 fac[0] = 1;
30 for(int i = 1; i <= 5e5; i++){
31 fac[i] = 1ll * fac[i - 1] * i % mod;
32 }
33 ll n,m;
34 cin >> n >> m;
35 for(int i = 1; i <= n; i++){
36 scanf("%lld",&a[i]);
37 b[i] = b[i - 1] + a[i];
38 }
39
40 q.push({a[1],1});
41 ll ans = 0;
42 ll sum = 0;
43 sum += 1;
44 for(int i = 1; i <= n - 1; i++){
45 q.push({a[i + 1],i + 1});
46 sum += ①;
47 sum %= mod;
48 if(②){
49 ans += qpow(2,i) * fac[n - i] % mod;
50 }else{
51 while(q.size()){
52 pii p = q.top();
53 if(b[p.i + 1] - p.x <= m) break;
54 sum -= ④;
55 sum %= mod;
56 sum += mod;
```

```
57 sum %= mod;
58 q.pop();
59 }
60 ans += sum * fac[n - i] % mod;
61 }
62 ans %= mod;
63 }
64
65 if(b[n] <= m) ans += ⑤;
66
67 cout << ((ans) % mod + mod) % mod << endl;
68 return 0;
69 }
```

38. ①处应填 (     )。

- A. pow(2,i)
- B. pow(2,i - 1)
- C. qpow(2,i)
- D. qpow(2,i - 1)

39. ②处应填 (     )。

- A. b[i] <= m
- B. b[i + 1] <= m
- C. sum <= m
- D. sum > m

40. ③处应填 (     )。

- A. less<int>
- B. greater<int>
- C. less<PII>
- D. greater<PII>

41. ④处应填 (     )。

- A. b[i + 1]
- B. b[i]
- C. qpow(2,p.y - 2)
- D. qpow(2,p.y - 1)

42. ⑤处应填 (     )。

- A. qpow(2,n - 1)
- B. qpow(2,n)
- C. qpow(2,n - 2)
- D. qpow(2,n - 2) \* fac[n]

## 提高组 CSP-S 2024 初赛模拟卷 7

扫码获取  
答案解析

一、单项选择题（共 15 题，每题 2 分，共计 30 分；每题有且仅有一个正确选项）

1. 以下哪个操作系统版本不属于 Linux 操作系统？（ ）

- A. Ubuntu      B. CentOS      C. RedHat      D. iOS

2. 对于已经排好序的数列，以下排序算法最快的是（ ）。

- A. 插入排序      B. 基数排序      C. 选择排序      D. 快速排序

3. 以下哪个排序从顺序存储改为链式存储时间复杂度会增加？（ ）

- A. 冒泡排序      B. 插入排序      C. 希尔排序      D. 堆排序

4. 一个有 54 个结点（根结点深度为 1）的完全四叉树，按前序遍历的顺序给结点从 1 开始编号，则第 30 号结点的父结点是第（ ）号。

- A. 5      B. 6      C. 7      D. 8

5. 有 5 枚硬币，第  $i$  枚硬币有  $\frac{p_i}{q_i}$  的概率抛出正面（剩余概率出现反面）。这些硬币抛

完后，正面数量超过反面数量的概率是多少？假设  $p_1 = 3$ ， $q_1 = 5$ ， $p_2 = 1$ ， $q_2 = 7$ ， $p_3 = 1$ ， $q_3 = 1$ ， $p_4 = 0$ ， $q_4 = 1$ ， $p_5 = 4$ ， $q_5 = 6$ 。（ ）

- A.  $\frac{1}{35}$       B.  $\frac{13}{35}$       C.  $\frac{7}{15}$       D.  $\frac{8}{21}$

6. 对二叉树 T 来说， $L(T)$ 表示 T 的根的左子树， $R(T)$ 表示 T 的根的右子树。二叉树 T 与二叉树 S 的大小关系定义如下。

- 首先，结点数  $Size$  的二叉树更小。
  - 其次，若结点数相同，则比较  $L(S)$ 与  $L(T)$ ，左子树更小的二叉树更小。
  - 若左子树也一样大，则比较  $R(S)$ 与  $R(T)$ ，右子树更小的二叉树更小。
- 二叉树 T 的中序序列定义如下。

- 先输出  $L(T)$ 的中序序列，并在外加一层括号，若  $L(T)$ 为空则跳过这步。
- 再输出一个 \*。
- 最后输出  $R(T)$ 的中序序列，并在外加一层括号，若  $R(T)$ 为空则跳过这步。

将所有二叉树按照上述规则从小到大排序，第 10 小的二叉树的中序序列是什么？

( )

A.  $((*)*(*)*)^*$       B.  $(*)*((*)^*)^*$       C.  $*(((*)^*)^*)^*$       D.  $(*(*)^*)^*(*)$

7. 下述哪个关于欧拉图和哈密顿图的说法正确？ ( )

A. 一张图不能同时称为欧拉图和哈密顿图

B. 欧拉图不一定含有欧拉回路

C. 只有有向图拥有欧拉回路

D. 无向连通图 G 是欧拉图，当且仅当 G 有零个、一个或两个奇数度的结点

8. 老师现在需要给 3 名学生分配 25 颗糖果，要求分给这 3 名学生分别不少于 1 颗、2 颗、3 颗糖果，问一共有多少种分法？ ( )

A. 120      B. 150      C. 210      D. 231

9. 在双向链表中定位删除一个元素的平均复杂度为 ( )。

A.  $O(1)$       B.  $O(n)$       C.  $O(\log n)$       D.  $O(n \log n)$

10. G 是一个非连通简单无向图，共有 28 条边，则该图至少有 ( ) 个顶点。

A. 10      B. 9      C. 8      D. 7

11. 堆排序建堆最快所需要的时间复杂度是 ( )。

A.  $O(1)$       B.  $O(n \log n)$       C.  $O(\log n)$       D.  $O(n)$

12. 学校里有 4 个盒子，第 1 个盒子有 3 朵花，第 2 个盒子有 3 朵花，第 3 个盒子有 3 朵花，第 4 个盒子有 6 朵花。同一个盒子内花的颜色相同，不同盒子内的花颜色不同。现要从中选出 4 朵花组成一束花，共有多少种方案？若两束花中相同颜色的花的数量都相同，则认为方案相同。( )

A. 33      B. 25      C. 30      D. 27

13. 树状数组是解决区间问题常用的数据结构，其原理是使用 C 数组去求 A 数组的前缀和。如下图所示： $C[1] = A[1]$ ， $C[2] = C[1] + A[2]$ ， $C[4] = C[2] + C[3] + A[4]$ 。现定义概念组成值和被组成值，例如  $C[2] = C[1] + A[2]$ ，那么  $C[1]$ 、 $A[2]$  都是  $C[2]$  的组成值，反过来说  $C[2]$  就是它俩的被组成值。可以发现，每一项 C 或者每一项 A 都最多成为别人的组成值一次。现请问  $C[11]$  的被组成值是多少？ ( )



```
03 using namespace std;
04
05 typedef long long ll;
06
07 const int N = 25, mod = 1e9 + 7;
08
09 ll a[N];
10
11 ll cal(ll a, ll b){
12 if(a < b) return 0;
13 ll x = 1;
14 for(ll i = a; i > a - b; i--){
15 x *= i;
16 x %= mod;
17 }
18 return x;
19 }
20
21 ll qmi(ll a, ll b){
22 ll s = 1;
23 while(b){
24 if(b & 1) s = s * a % mod;
25 a = a * a % mod;
26 b >>= 1;
27 }
28 return s;
29 }
30
31 int main(){
32 int n, ll m;
33 cin >> n >> m;
34 for(int i = 0; i < n; i++){
35 cin >> a[i];
36 }
37
38 ll s = 1;
39
40 for(int i = 1; i <= n - 1; i++){
41 s = s * i % mod;
42 }
43 s = qmi(s, mod - 2);
44 }
```

```
45 ll ans = 0;
46 for(int i = 0; i < (1 << n); i++){
47 ll h = n + m - 1; int sign = 1;
48 for(int j = 0; j < n; j++){
49 if(i & (1 << j)){
50 h -= a[j] + 1;
51 sign = ~sign + 1;
52 }
53 }
54 h = cal(h, n - 1);
55 ans += h * s * sign % mod;
56 ans %= mod;
57 }
58
59 cout << (ans % mod + mod) % mod << endl;
60
61 return 0;
62 }
```

### ■ 判断题

16. 将第 41 行改成 `s *= i % mod`, 结果不会改变。 ( )
17. 将第 51 行改成 `sign = 1 - sign`, 结果不会改变。 ( )
18. 在不考虑 `mod` 的情况下, 当 `n` 固定时, `m` 增大, 最后输出的值一定增大。 ( )
19. 该程序运用的思想是容斥思想。 ( )

### ■ 选择题

20. 当输入 4 4 3 3 3 6 时, 程序输出 ( )。
- A. 30                      B. 31                      C. 32                      D. 33

21. 该程序的时间复杂度是 ( )。

A.  $O(n \log n)$                       B.  $O(2^n n)$                       C.  $O(2^n \log n)$                       D.  $O(2^n (n + \log n))$

(2)

```
01 #include<iostream>
02 using namespace std;
03
04 int phi[40005];
05
```

```

06 int get(int n){
07 int sum = n;
08 for(int i = 2;i < n; i++){
09 if(n % i == 0){
10 sum -= sum / i;
11 while(n % i == 0) n /= i;
12 }
13 }
14 if(n > 1) sum -= sum / n;
15 return sum;
16 }
17
18 int main(){
19 int n;
20 scanf("%d",&n);
21 if(n == 1){
22 return 0;
23 }
24 for(int i = 2;i <= n; i++){
25 phi[i] = i;
26 }
27 for(int i = 2;i <= n; i++){
28 if(phi[i] != i) continue;
29 for(int j = i;j <= n;j += i)
30 phi[j] = phi[j] / i * (i-1);
31 }
32 int ans = 1;
33 for(int i = 1;i < n; i++) ans += phi[i];
34
35 int ans2 = 0;
36 for(int i = 2;i <= n - 1; i++) ans2 += get(i);
37
38
39 printf("%d %d",ans * 2 + 1,ans2 * 2 + 3);
40 }

```

### ■ 判断题

22. 求解 phi 数组的算法的时间复杂度为  $O(n\log n)$ 。 (      )
23. 将第 8 行代码改成 `for(int i = 2;i * i <= n; i++)` 不会导致答案错误。 (      )
24. ( 3 分 ) 最后输出的两个值总是相等。 (      )

## ■ 选择题

25. 求解 ans2 值的算法的时间复杂度为 ( )。

- A.  $O(n^2)$       B.  $O(n^2\sqrt{n})$       C.  $O(n^2\log n)$       D.  $O(n\log n)$

26. 若输入 20, 程序输出 ( )。

- A. 241 241      B. 239 241      C. 255 257      D. 205 205

27. 当输入 35 时, phi[36]与 get(36)分别等于 ( )。

- A. 12 12      B. 0 0      C. 11 12      D. 0 12

(3)

```
01 #include<iostream>
02 #include<cstring>
03 #include<algorithm>
04 using namespace std;
05 #define MOD 1000000007
06 char s[1200000];
07 int nt[1200000];
08 int jp[21][1200000];
09 int main() {
10 scanf("%s", s+1);
11 nt[1] = 0;
12 int l = strlen(s+1);
13 for(int i = 2; i <= l; ++i) {
14 int j = nt[i - 1];
15 while(j && s[i] != s[j + 1]) j = nt[j];
16 if(s[i] == s[j + 1]) nt[i] = j + 1;
17 else nt[i] = 0;
18 jp[0][i] = nt[i];
19 }
20 for(int j = 1; j < 20; ++j)
21 for(int i = 1; i <= l; ++i)
22 jp[j][i] = jp[j - 1][jp[j - 1][i]];
23 int ans = 1;
24 for(int i = 2; i <= l; ++i) {
25 int tt = i;
26 for(int j = 19; j >= 0; --j)
```

```
27 if(jp[j][tt] * 2 > i) tt = jp[j][tt];
28 int gg = 0;
29 for(int j = 19; j >= 0; --j)
30 if(jp[j][tt]) gg += 1 << j, tt = jp[j][tt];
31 ans = 1ll * ans * (gg + 1) % MOD;
32 }
33 printf("%d\n", ans);
34 return 0;
35 }
```

### ■ 判断题

28. (3分) 当输入的字符串为 abcaabcaab 时, nt[10]为5。 ( )
29. 当输入的字符串为 abcaabcaab 时, 第 15 行中的  $j = nt[j]$ —共执行 1 次。 ( )
30. 该程序最后的输出结果可能为 0。 ( )

### ■ 选择题

31. 当输入为 aaaaa 时, 程序的输出为 ( )。
- A. 12                      B. 24                      C. 36                      D. 48
32. (4分) 若不考虑% MOD, 字符串长度至少为 ( ), 程序的输出大于 100。
- A. 5                        B. 6                        C. 7                        D. 8

### 三、完善程序 (单选题, 每小题 3 分, 共计 30 分)

- (1) 一共有  $n$  个数, 第  $i$  个数  $x_i$  可以取  $[a_i, b_i]$  中的任意值。设  $S = \sum x_i^2$ , 求  $S$  的种类数。  
 $1 \leq n, a_i, b_i \leq 100$ 。考虑极限情况下  $S$  最大为 1 000 000, 使用 DP 解题。题目时限 1s, 空间限制为 256 MB。

```
01 #include <bits/stdc++.h>
02 #define maxn 1000005
03 using namespace std;
04 bitset<maxn> bit1, bit2;
05 int a[105], b[105];
06 int dp[N][
07 int main() {
08 int n;
09 scanf("%d", &n);
10 for (int i = 1; i <= n; i++) {
```

```
11 scanf("%d%d", &a[i], &b[i]);
12 }
13 for (int i = 1; i <= n; i++) {
14 int l = a[i], r = b[i];
15 for (int j = l; j <= r; j++) {
16 if (i == 1)
17 ①;
18 else
19 ②;
20 }
21 if (i == 1)
22 bit2 = bit1;
23 bit1 = bit2;
24 ③;
25 }
26 printf("%d\n", ④);
27 }
```

33. ①处应填 ( )。

- |                    |                    |
|--------------------|--------------------|
| A. bit1.set(j * j) | B. bit2.set(j * j) |
| C. bit1.set(i * i) | D. bit2.set(i * i) |

34. ②处应填 ( )。

- |                            |                            |
|----------------------------|----------------------------|
| A. bit2 ^= bit1 << (j * j) | B. bit2 += bit1 << (j * j) |
| C. bit2  = bit1 << (j * j) | D. bit2 &= bit1 << (j * j) |

35. ③处应填 ( )。

- |                 |                 |
|-----------------|-----------------|
| A. bit1.reset() | B. bit2.reset() |
| C. bit2.set(0)  | D. bit1.set(0)  |

36. ④处应填 ( )。

- |                 |                  |
|-----------------|------------------|
| A. bit1         | B. bit2.count()  |
| C. bit1.count() | D. ~bit1.count() |

(2) 有  $n$  个人, 每个人的空闲时间为  $[a_i, b_i]$ , 现需要将  $n$  个人分至  $k$  个组别中。每个组的成员能一起玩的时间是所有入空闲时间的交集。现希望知道怎么分组才能使得组成员一起玩的时间总和最大。需要注意的是, 每个组一起玩的时间至少要大于一分钟,

每个组的成员至少为 1 位。

解题思路：

将题目中的空闲时间区间分为两种——可以包含其他区间的大区间，以及小区间。对于包含其他区间的大区间，最优的情况即是单独一组或者与其被包含的区间一组。所以可以先将这些大区间去掉，观察剩余区间的性质。

```
01 #include<iostream>
02 #include<cstring>
03 #include<algorithm>
04 #include<map>
05 #include<vector>
06
07 using namespace std;
08
09 const int N = 5e3 + 10;
10 int n,k;
11 struct node{
12 int l,r;
13 bool operator < (const node &p)const{
14 if(l == p.l) return r > p.r;
15 return l < p.l;
16 }
17 }a[N],b[N];
18
19 int seg[N];
20
21 int dp[N][N]; //前 i 个数用了 j 个片段
22
23 int q[N][N],tail[N],head[N];
24
25 int main(){
26 cin >> n >> k;
27 for(int i = 1; i <= n; i++){
28 cin >> a[i].l >> a[i].r;
29 }
30
31 sort(a + 1,a + n + 1);
32
33 int cnt = 0,tot = 1;
34 int maxr = 0x3f3f3f3f;
35 for(int i = n; i >= 1; i--){
```

```

36 if(maxr <= a[i].r){
37 seg[tot++] = a[i].r - a[i].l;
38 }else{
39 b[++cnt] = a[i];
40 ①;
41 }
42 }
43 memset(dp, -0x3f, sizeof dp);
44 sort(b + 1, b + cnt + 1);
45 ②;
46
47 dp[0][0] = 0;
48 for(int i = 1; i <= k; i++) tail[i] = -1;
49 for(int i = 1; i <= cnt; i++){
50 for(int l = k; l >= 1; l--){
51 while(dp[q[l]][tail[l]] - 1][l - 1] + b[q[l]][tail[l]]].
52 r < ③ 88 tail[l] >= head[l]) --tail[l];
53 q[l][++tail[l]] = i;
54 while(④ 88 head[l] <= tail[l]){
55 ++head[l];
56 }
57 dp[i][l] = max(dp[i][l], dp[q[l]][head[l]] - 1][l - 1] + ⑤);
58 }
59 }
60 int sum = 0;
61 int maxn = dp[cnt][k];
62 for(int i = 1; i <= tot 88 i <= k; i++){
63 sum += seg[i];
64 maxn = max(maxn, ⑥);
65 }
66
67 if(maxn > 0) cout << maxn << endl;
68 else cout << 0 << endl;
69
70
71 return 0;
72 }

```

37. ①处应填 (      )。

A. maxr = a[i].l

B. seg[++tot] = maxr - a[i].l

C. `maxr = a[i].r`

D. `--tot`

38. ②处应填 (      )。

A. `sort(seg + 1, seg + tot, greater<int>())`

B. `sort(seg + 1, seg + tot + 1, less<int>())`

C. `sort(seg + 1, seg + tot + 1, greater<int>())`

D. `sort(seg, seg + tot, less<int>())`

39. ③处应填 (      )。

A. `dp[i - 1][l - 1] + b[i].r`

B. `dp[i - 1][l - 1] + b[i].l`

C. `dp[i - 1][l - 1] + b[i - 1].r`

D. `dp[i - 1][l - 1] + b[i - 1].l`

40. ④处应填 (      )。

A. `b[q[l - 1][head[l - 1]]].r <= b[i].l`

B. `b[q[l - 1][tail[l - 1]]].r <= b[i].l`

C. `b[q[l][tail[l]]].r <= b[i].l`

D. `b[q[l][head[l]]].r <= b[i].l`

41. ⑤处应填 (      )。

A. `b[q[l][head[l]]].l - b[i].l`

B. `b[q[l - 1][head[l - 1]]].r - b[i - 1].l`

C. `b[q[l - 1][head[l - 1]]].r - b[i].l`

D. `b[q[l][head[l]]].r - b[i].l`

42. ⑥处应填 (      )。

A. `sum + dp[cnt][k]`

B. `sum + dp[cnt][k - i]`

C. `sum + dp[k - i][cnt]`

D. `sum + dp[i][cnt]`

## 提高组 CSP-S 2024 初赛模拟卷 8

扫码获取  
答案解析

一、单项选择题（共 15 题，每题 2 分，共计 30 分；每题有且仅有一个正确选项）

1. 下列设备中，属于 OSI 七层模型中数据链路层的是（ ）。  
A. 以太网线      B. 路由器      C. 交换机      D. 光猫
2. 在 NOI Linux 终端系统中，下列哪个命令可以将 a.cpp 重命名为 b.cpp？（ ）  
A. mv a.cpp b.cpp      B. cp < a.cpp > b.cpp  
C. rm A.cpp b.cpp      D. echo > a.cpp < b.cpp
3. 某算法的执行时间由以下递归式定义： $T(n) = 2T\left(\frac{n}{2}\right) + O(n^c)$  ( $c > 1$ )。该递归式的时间复杂度是（ ）。  
A.  $O(n \log n)$       B.  $O(n^{\frac{1}{c}})$       C.  $O(n^c \log n)$       D.  $O(n)$
4. 以下对数据结构或算法的表述中不恰当的一项是（ ）。  
A. 栈是一种后进先出的数据结构  
B. 平衡树可以在  $O(n)$  时间内完成遍历  
C. 随机访问长度为  $n$  的链表中一个元素的期望复杂度是  $O(1)$   
D. 二叉堆是一棵完全二叉树
5. 执行以下 C++ 代码后，z 的值是（ ）。  

```
int z = (unsigned long long)(-5) % 5
```

  
A. 0      B. 1      C. 2      D. 3
6. 一棵大小为  $n$  的树恰好有两个重心的必要不充分条件是（ ）。  
A. 这棵树有奇数个结点  
B. 这棵树有偶数个结点  
C. 这棵树存在两个结点，使得删去它们后剩下的所有连通块的大小不超过  $\left\lfloor \frac{n}{2} \right\rfloor$   
D. 这棵树可以完成黑白染色

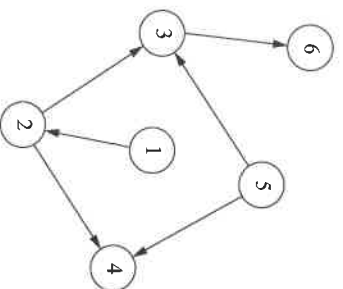
7. 关于  $x, y$  的整系数方程  $ax + by = d$  的解, 下列说法中正确的是 ( )。
- A. 当且仅当  $d = \gcd(a, b)$  时, 该方程存在一组整数解
- B. 该方程要么有无数个整数解, 要么无解
- C. 若  $x_0, y_0$  为该方程的一组解, 则该方程的通解为  $x = x_0 - kb, y = y_0 + ka, k \in \mathbb{Z}$
- D. 在使用 `exgcd` 算法求解上述问题的过程中, 若  $|a|, |b| \leq 10^9$ , 则计算时的中间结果需要使用 `long long` 保存

8. 有 8 个结点, 每个点度数不超过 3 的树最多有 ( ) 个叶子结点。
- A. 3                      B. 4                      C. 5                      D. 以上答案都不对
9. 以下关于算法复杂度的描述, 不正确的是 ( )。
- A. 基于比较的排序复杂度可以低于  $O(n \log n)$
- B. 对一个有  $n$  个顶点、 $m$  条边的带权有向图用 Dijkstra 算法计算单源最短路时, 若使用斐波那契堆进行优化, 则复杂度为  $O(n \log n + m)$
- C. RMQ (区间最值查询) 问题的最优在线算法时间复杂度是  $O(n \log n + q)$
- D. 普通线段树的空间复杂度是  $O(n)$

10. 定义一个数  $x$  是优美的, 当且仅当  $x$  的二进制表示下 0 比 1 多, 问 0 ~ 255 中有多少个数是优美的? ( )

A. 64                      B. 65                      C. 127                      D. 128

11. 下图合法的拓扑序数量是 ( )。



A. 9                      B. 12                      C. 8                      D. 7

12. 定义两个字符串 A、B 的编辑距离如下。

设 A 和 B 是两个字符串，编辑距离指使字符串 A 转换为字符串 B 所需的最少字符操作次数。这里所说的字符操作共有 3 种：

- 删除一个字符；
- 插入一个字符；
- 将一个字符改为另一个字符。

字符串 platelets 与 planets 的编辑距离是 ( )。

- A. 3                      B. 4                      C. 5                      D. 6

13. 现在用如下代码计算  $\frac{1}{2}((\sum a_i)^2 - \sum a_i^2)$ ，其中 a 的下标从 0 开始，长度为  $2^n$ ，其时间复杂度为 ( )。

```
int Sol(int *a, int n) {
 int sum = 0;

 for (int o = 1; o < (1 << n); o <= 1) {
 for (int i = 0; i < (1 << n); i += (o << 1)) {
 for (int j = 0; j < o; j++)
 for (int k = 0; k < o; k++)
 sum += a[i + j] * a[i + k + o];
 }
 }
 return sum;
}
```

- A.  $O(4^n n)$                       B.  $O(3^n n^2)$                       C.  $O(4^n)$                       D.  $O(8^n)$

14. 学生需要将编号为 1, 2, 3, 4, 5, ... 的方块依次放入盒子中，每次放入方块时，需要保证盒子中其余方块（不包括当前放入的方块）的编号之和为完全平方数。当有 5 个盒子时，最多可以放置 ( ) 个方块。

- A. 8                      B. 10                      C. 12                      D. 13

15. 甲乙二人各有两枚石头，在一个回合中，一人抛硬币，如果正面向上，则将一枚石头交给对方，交不出来就输了，如果反面向上则什么都不做。甲乙二人轮流进行游戏，甲先手，甲获胜的概率是 ( )。

- A.  $\frac{4}{7}$                       B.  $\frac{35}{81}$                       C.  $\frac{11}{27}$                       D.  $\frac{37}{81}$

二、阅读程序（程序输入不超过数组或字符串定义的范围；判断题正确填√，错误填×；除特殊说明外，判断题每题 1.5 分，选择题每题 3 分，共计 40 分）

(1)

```
01 #include <bits/stdc++.h>
02 using namespace std;
03 int a[1 << 10], b[1 << 10], c[1 << 10], d[1 << 10], n;
04 int main() {
05 scanf("%d", &n);
06 for (int i = 0; i < (1 << n); i++)
07 scanf("%d", &a[i]), d[i] = a[i];
08 for (int s = 0; s < (1 << n); s++)
09 for (int s2 = 0; s2 < (1 << n); s2++)
10 if ((s & s2) == s2)
11 b[s] += a[s2];
12 for (int s = 0; s < (1 << n); s++) {
13 int s2 = s;
14 while (s2)
15 c[s] += a[s2], s2 = (s2 - 1) & s;
16 }
17 for (int i = 0; i < n; i++)
18 for (int s = 0; s < (1 << n); s++)
19 if ((s >> i) & 1)
20 d[s] += d[s ^ (1 << i)];
21 for (int i = 0; i < (1 << n); i++)
22 printf("%d ", b[i]);
23 puts("");
24 for (int i = 0; i < (1 << n); i++)
25 printf("%d ", c[i]);
26 puts("");
27 for (int i = 0; i < (1 << n); i++)
28 printf("%d ", d[i]);
29 puts("");
30 return 0;
31 }
```

注：输入的  $n$  满足  $1 \leq n \leq 10$ ,  $1 \leq a_i \leq 10^4$ 。

## ■ 判断题

16. 输出中的第 2 行一定与第 1 行不同。 ( )
17. 输出中的第 3 行不一定与第 1 行相同。 ( )
18. 把第 14 行中的 s2 改为 s2>=0, 程序不会进入死循环。 ( )
19. (2 分) 把第 17 行中的 for(int i = 0; i < n; i++) 改成 for(int i = n-1; i >= 0; i--) 后, 程序的输出结果不变。 ( )

## ■ 选择题

20. 当输入为 0 1 时, 第 2 行输出结果为 ( )。
- A. 1                      B. 0                      C. 2                      D. -1
21. 当输入为 3 1 2 3 4 5 6 7 8 时, 第 2 行输出结果为 ( )。
- A. 1 2 3 4 5 6 7 8                      B. 1 3 4 10 6 11 16 36
- C. 0 2 3 9 5 10 15 35                      D. 0 2 3 9 5 13 15 35

(2)

```
01 #include <bits/stdc++.h>
02
03 using namespace std;
04
05 long long a[10000];
06
07 int main() {
08 int n, m, k;
09 cin >> n >> m >> k;
10 a[0] = 1;
11 while (k--) {
12 long long sum[100] = {};
13 for (int i = 0; i < n; i++)
14 for (int j = 0; j < m; j++)
15 sum[j] = a[i * m + j] += sum[j];
16 long long S = 0;
17 for (int i = 0; i < m; i++)
18 swap(sum[i], S), S += sum[i];
19 for (int i = 0; i < n; i++)
20 for (int j = 0; j < m; j++)
21 a[i * m + j] += sum[j];
```

```
22 }
23 cout << a[n * m - 1] << endl;
24 }
```

注:  $n, m, k$  都是正整数, 程序运行中不会发生 long long 溢出。

### ■ 判断题

22. 当  $n \times m \leq 10\,000$  时, 一定不会发生数组越界。 ( )
23. 删掉第 16~21 行, 输出一定不会增大。 ( )
24. (2 分) 当  $n, m, k$  中任意一个增大而另外两个不变时, 输出一定会增大。 ( )

### ■ 选择题

25. 当输入为 1 2 10 时, 输出为 ( )。
- A. 1                      B. 10                      C. 55                      D. 1024
26. 当输入为 20 15 10 时, 记输出为 X; 当输入为 15 20 10 时, 记输出为 Y。那么 X 和 Y 的大小关系为 ( )。
- A.  $X < Y$                   B.  $X = Y$                   C.  $X > Y$                   D. 不能确定
27. 当输入为 100 100 3 时, 输出为 ( )。
- A. 49995000              B. 50005000              C. 50015001              D. 50025003

(3)

```
01 #include <bits/stdc++.h>
02
03 using namespace std;
04
05 int foobar(vector<int> A, int k) {
06 if (A.size() < 5) {
07 sort(A.begin(), A.end());
08 return A[k];
09 }
10 vector<int> B;
11 for (int i = 0; i + 4 < A.size(); i += 5) {
12 sort(A.begin() + i, A.begin() + i + 5);
13 B.push_back(A[i + 2]);
14 }
```

```

15 int value = foobar(B, B.size() / 2);
16 vector<int> left, right;
17 for (int v : A) {
18 if (v < value) left.push_back(v);
19 if (v > value) right.push_back(v);
20 }
21 if (k < left.size()) return foobar(left, k);
22 if (k < A.size() - right.size()) return value;
23 return foobar(right, k - (A.size() - right.size()));
24 }
25
26 int main() {
27 ios::sync_with_stdio(false), cin.tie(nullptr);
28 int n, k;
29 cin >> n >> k;
30 vector<int> A(n);
31 for (int i = 0; i < n; i++) cin >> A[i];
32 cout << foobar(A, k) << endl;
33 }
```

注：假设  $0 \leq k \leq n \leq 10^6$ 。

### ■ 判断题

28. 把第 15 行中的 `B.size()` / 2 改成 `A.size()` / 10，效果不变。 ( )
29. 把第 18 行中的 `v < value` 改为 `v <= value`，不影响程序的正确性。 ( )
30. 把第 22 行中的 `k < A.size() - right.size()` 改成 `A.size() - right.size() - k > 0`，效果不变。 ( )

### ■ 选择题

31. 当输入为 6 3 3 1 5 5 2 2 时，输出为 ( )。  
A. 1                      B. 2                      C. 3                      D. 5
32. 最坏情况下，该程序的时间复杂度为 ( )。  
A.  $O(n)$                   B.  $O(n \log n)$               C.  $O(n^{\log_5 6})$               D.  $O(n^2)$
33. 假设删掉第 12 行，最坏情况下该程序的时间复杂度为 ( )。  
A.  $O(n \log n)$               B.  $O(n^{\log_{10} 11})$               C.  $O(n^2)$                       D. 以上都不对

## 三、完善程序 (单选题, 每小题 3 分, 共计 30 分)

## (1) 最长超集子序列

给定一个  $0 \sim 2^n - 1$  ( $1 \leq n \leq 20$ ) 的排列  $P$ , 定义超集子序列  $s_{i_1}, s_{i_2}, s_{i_3}, \dots, s_{i_k}$  需要满足  $P_{s_{i_j}} \subseteq P_{s_{i_{j+1}}}$  ( $1 \leq j \leq k-1$ ), 即  $(P_{s_{i_j}} | P_{s_{i_{j+1}}}) = P_{s_{i_{j+1}}}$ , 求出最长超集子序列的长度。

提示: 设  $dp_i$  表示以  $P_i$  结尾的最长超集子序列的长度, 转移为  $dp_i = \max\{dp_j\} + 1$  ( $P_j \subseteq P_i$ ), 直接转移复杂度为  $O(3^n)$ , 无法通过。动态维护辅助数组  $g_{A,B}$  表示

$\max\{dp_j\}$ , 其中  $j$  要满足  $P_j$  在二进制表示下的高  $\left\lfloor \frac{n}{2} \right\rfloor$  为 A、低  $\left\lfloor \frac{n}{2} \right\rfloor$  为 B 的子集, 用

公式表达为:

$$g_{A,B} = \max\{dp_j\} \quad (C \subseteq B, P_j = A \times 2^{\left\lfloor \frac{n}{2} \right\rfloor} + C)$$

试补全程序。

```

01 #include <bits/stdc++.h>
02
03 using namespace std;
04
05 int n, P[1 << 20], dp[1 << 20], g[1 << 10][1 << 10];
06
07 void foobar(int& a, int b) {
08 if (①) a = b;
09 }
10 int main() {
11 ios::sync_with_stdio(false), cin.tie(nullptr);
12 cin >> n;
13 for (int i = 0; i < 1 << n; i++) cin >> P[i];
14 for (int i = 0; i < 1 << n; i++) {
15 int A = P[i] >> n / 2, B = P[i] & (②);
16 for (int subset = A;; --subset &= A) {
17 foobar(dp[i], g[subset][B] + 1);
18 if (subset == 0) break;
19 }
20 for (int superset = B;; ③) {
21 foobar(④, dp[i]);
22 if (superset == ⑤) break;
23 }
24 }

```

```
25 int ans = 0;
26 for (int i = 0; i < 1 << n; i++) foobar(ans, dp[i]);
27 cout << ans << endl;
28 }
```

34. ①处应填 ( )。

- A.  $a < b$                       B.  $a > b$                       C.  $a = b$                       D.  $a != b$

35. ②处应填 ( )。

- A.  $1 << n$                       B.  $(1 << n) - 1$   
C.  $(1 << n) - (1 << n / 2)$                       D.  $(1 << n / 2) - 1$

36. ③处应填 ( )。

- A. `++superset &= B`                      B. `++superset |= B`  
C. `--superset &= B`                      D. `--superset |= B`

37. ④处应填 ( )。

- A. `g[A][superset] + 1`                      B. `g[B][superset] + 1`  
C. `g[superset][A] + 1`                      D. `g[superset][B] + 1`

38. ⑤处应填 ( )。

- A.  $\emptyset$                       B.  $(1 << n) - 1$   
C.  $(1 << n) - (1 << n / 2)$                       D.  $(1 << n / 2) - 1$

## (2) 有限分数计数

给定正整数  $n$  ( $1 \leq n \leq 10^7$ ), 求出有序整数对  $(x, y)$  的个数, 满足  $1 \leq x, y \leq n$ , 且  $\frac{x}{y}$  可以表示为十进制有限小数。

提示:  $\frac{x}{y}$  可以表示为十进制有限小数的条件为, 先把  $y$  中所有 2 和 5 的因子去掉得

到  $y'$ ,  $\frac{x}{y'}$  必须是整数。我们可以枚举  $y'$ , 快速算出有多少对  $(x, y)$  合法。

试补全程序。

```
01 #include <bits/stdc++.h>
02
```

```

03 using namespace std;
04
05 int main() {
06 int n;
07 cin >> n;
08 vector<int> num;
09 for (int A = 1; A <= n; A *= 2)
10 for (int B = 1; A * B <= n; ①)
11 num.push_back(A * B);
12 ②;
13 long long ans = 0;
14 for (int i = 1; i <= n; i++)
15 if (③) {
16 while (!num.empty() && num.back() > ④) num.pop_back();
17 ans += ⑤;
18 }
19 cout << ans << '\n';
20 }

```

39. ①处应填( )。

A. B++

$$B.B^* = 2$$

C.B. \* 5

D.B \*= 10

40. ②处应填( )。

```
A.sort(num.begin(), num.end(), less<int>())
```

```
B.sort(num.begin(), num.end(), greater<int>())
```

```
C.sort(num.rbegin(), num.rend())
```

```
D.sort(num.rbegin(), num.rend(), less<int>())
```

41. ③处应填( )。

A.  $i \% 2 == 0 \ \&\& \ i \% 5 == 0$       B.  $i \% 2 == 0 \ || \ i \% 5 == 0$

C.  $i \% 2 != 0 \&\& i \% 5 != 0$       D.  $i \% 2 == 0 || i \% 5 != 0$

42. ④处应填( )。

A.i

B. n

C. n / i

D.n - 111 \* i \* i

43. ⑤处应填( )。

A.num.size()

```
B.1LL * num.size()* i
```

```
C.num.size()* n / i
```

```
D.n / i * num.size()
```

## 提高组 CSP-S 2024 初赛模拟卷 9



扫码获取  
答案解析

一、单项选择题（共 15 题，每题 2 分，共计 30 分；每题有且仅有一个正确选项）

1. 在  $p$  进制下 ( $9 \leq p \leq 16$ ) 有如下等式成立：  $7*8=44$ ，则该进制下  $52*74=(\quad)$ 。  
A. 2b88                      B. 6365                      C. 3848                      D. 28b5
2. 在 8 位二进制补码中，10101101 表示的数是十进制下的  $(\quad)$ 。  
A. 45                          B. -45                          C. -83                          D. 173
3. 后缀表达式  $9 \quad 3 \quad 1 \quad - \quad 3 \quad * \quad + \quad 10 \quad 2 \quad / \quad +$  的计算结果为  $(\quad)$ 。  
A. 20                          B. 1030                          C. 2                              D. 15
4. 7 个同学坐在一排，由于现在要考试，为了避免同学们在自己的桌子上留下小抄作弊，老师打算打乱所有人的位置，以保证每个人都不坐在自己原来的位置，那么一共有  $(\quad)$  种打乱方式。  
A. 265                          B. 1854                          C. 720                          D. 1440
5. 若某算法的计算时间表示为递推关系  
$$T(n) = 2T(n/2) + n \log_2 n$$
则该算法的时间复杂度为  $(\quad)$ 。  
A.  $O(n \log_3^3 n)$               B.  $O(n \log_2^2 n)$               C.  $O(n \log_2 n)$               D.  $O(n)$
6. 若让元素 1, 2, 3, 4, 5 依次进栈，则出栈次序不可能出现  $(\quad)$  的情况。  
A. 5, 4, 3, 2, 1              B. 2, 1, 5, 4, 3              C. 4, 3, 1, 2, 5              D. 1, 2, 5, 4, 3
7. NOI Linux 2 是 CCF NOI 系列赛事的标准竞赛环境之一。以下终端命令中，能够显示当前目录名称的是  $(\quad)$ 。  
A. cat                          B. ls                              C. man                          D. pwd
8. 以下程序段的作用是求出 1 至  $n$  的所有  $(\quad)$ 。  
A. 素数个数                      B. 整数的最大因数  
C. 整数的因数个数              D. 整数的  $\phi(i)$

```

for (i=1;i<=n;i++) a[i]=i;
for (i=2;i<=n;i++)
 if (i==a[i])
 {
 for (j=1;j*i<=n;j++)
 a[j*i]=a[j*i]/i*(i-1);
 }

```

9. 给定地址区间为 0~12 的哈希表，哈希函数为  $h(x) = x^2 \% 13$ ，采用线性探查的冲突解决策略（若出现冲突情况，会往后探查第一个空的地址存储；若地址 12 冲突了，则从地址 0 重新开始探查）。哈希表初始为空表，依次存储(3,4,5,6,7,8,9,10)后，10 存储在哈希表的哪个地址中？（ ）

A. 0      B. 1      C. 9      D. 10

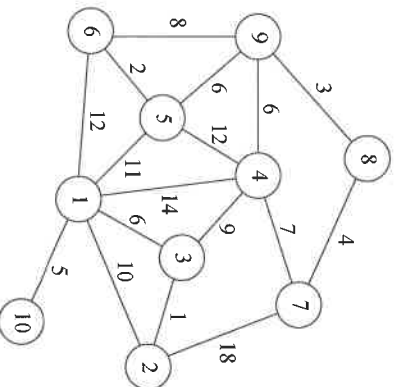
10. 令根结点的高度为 1，则一棵含有 2023 个结点的二叉树和一棵含有 2023 个结点的三叉树的高度分别至少为（ ）。

A. 11, 8      B. 11, 7      C. 10, 7      D. 10, 8

11. 以下选项中哪一个的期望时间复杂度与其他三个不同？（ ）

- A. 包含  $n$  个元素的 set 容器使用 lower\_bound 查询返回第一个大于某个数的迭代器
- B. 包含  $n$  个元素的 vector 使用 erase 删除  $n/2$  位置的元素
- C. 从包含  $n$  个元素的数组中构建一个小顶堆
- D. 求包含  $n$  个结点的树的直径

12. 该无向图的最小生成树的权值为（ ）。



A. 39      B. 38      C. 47      D. 42

13. 以下说法中不正确的是 ( )。

- A. 无向图具有欧拉回路的充分必要条件是每个点的度数均为奇数
- B. 在稠密图上, 相较于堆优化的 Dijkstra 算法, SPFA 表现更优
- C. 无向图是二分图, 当且仅当图上不存在奇数长度的环
- D. 树的重心不一定唯一

14. 掷 7 次质地均匀的正方体骰子, 出现和为 11 的概率可表示为  $\frac{x}{6^7}$ , 则  $x$  的值为 ( )。

- A. 84
- B. 210
- C. 121
- D. 240

15. 设循环队列中数组的下标范围是  $1 \sim m$ , 其头尾指针分别为  $\text{front}$  和  $\text{rear}$ ,  $\text{front}$  指向队列的第一个元素,  $\text{rear}$  指向最后一个元素位置的下一个位置, 则其元素个数为 ( )。

- A.  $\text{rear} - \text{front}$
- B.  $\text{rear} - \text{front} + 1$
- C.  $(\text{rear} - \text{front} + 1) \text{ MOD } m$
- D.  $(\text{rear} - \text{front} + m) \text{ MOD } m$

二、阅读程序 (程序输入不超过数组或字符串定义的范围; 判断题正确填√, 错误填×; 除特殊说明外, 判断题每题 1.5 分, 选择题每题 3 分, 共计 40 分)

(1)

```
01 #include<iostream>
02 #include<algorithm>
03 #define Type double
04 using namespace std;
05
06 const int maxn = 100005;
07 struct point
08 {
09 Type x, y;
10 point() {}
11 point(Type x, Type y) : x(x), y(y) {}
12 point operator - (const point b) const
13 {
14 return point(x - b.x, y - b.y);
15 }
16 void input()
17 {
18 cin >> x >> y;
19 }
```

```
20 } pnt[maxn], res[maxn];
21
22 Type cross(point a, point b)
23 {
24 return a.x * b.y - a.y * b.x;
25 }
26
27 int solve(int n)
28 {
29 sort(pnt, pnt + n, [g](point a, point b)
30 {
31 return a.x==b.x? a.y<b.y : a.x<b.x;
32 });
33 int m = 0;
34 for(int i = 0; i < n; ++i)
35 {
36 while(m > 1 && cross(res[m-1] - res[m-2],
37 pnt[i] - res[m-1]) <= 0)
38 --m;
39 res[m++] = pnt[i];
40 }
41 int k = m;
42 for(int i = n - 2; i >= 0; --i)
43 {
44 while(m > k && cross(res[m-1] - res[m-2],
45 pnt[i] - res[m-1]) <= 0)
46 --m;
47 res[m++] = pnt[i];
48 }
49 if(n > 1) --m;
50 return m;
51 }
52
53 int main()
54 {
55 int n;
56 cin>>n;
57 for(int i=0; i<n; ++i) pnt[i].input();
58 cout << solve(n) << endl;
```

假设输入的所有数均为整数，且绝对值都不超过 100 000，以下是 3 个输入样例。

| 输入样例 1: | 输入样例 2: | 输入样例 3: |
|---------|---------|---------|
| 5       | 11      | 6       |
| 0 0     | 0 4     | 0 0     |
| 0 2     | 2 8     | 4 4     |
| 2 2     | 4 4     | 0 4     |
| 2 1     | 4 1     | 2 2     |
| 2 0     | 8 2     | 2 0     |
|         | 8 8     | 4 0     |
|         | 9 6     |         |
|         | 14 0    |         |
|         | 12 4    |         |
|         | 13 8    |         |
|         | 12 10   |         |

■ 判断题

- 16. 在输入样例 1 的情况下，结果为 5。 ( )
- 17. 将第 3 行中的 double 修改为 int，所有合法输入的结果均不会发生变化。 ( )
- 18. 将第 36 行和第 43 行中的 <= 均修改为 <，输入样例 1 的输出结果不会发生变化。 ( )
- 19. 该算法的时间复杂度为  $O(n\log n)$ 。 ( )

■ 选择题

- 20. 在输入样例 2 的情况下，结果为 ( )。  
A. 4                      B. 5                      C. 6                      D. 7
- 21. (4 分) 在输入样例 3 的情况下，第 37 行和第 44 行的运行次数加起来一共为 ( )。  
A. 5                      B. 6                      C. 7                      D. 8

(2)

```
01 #include<iostream>
02 #include<algorithm>
03 using namespace std;
04
```

```
05 const int maxn = 2e5+7;
06
07 int func1(const char str[], int n)
08 {
09 int k=0,i=0,j=1;
10 for(; j<n; j++)
11 {
12 while(k<n&&str[i+k]==str[j+k])
13 {
14 k++;
15 }
16 if(str[i+k]>str[j+k])
17 i=j;
18 k=0;
19 }
20 return i;
21 }
22
23 int func2(const char str[],int n)
24 {
25
26 int i=0,j=1,k=0;
27 while(i<n && j<n && k<n)
28 {
29 if(str[i+k]==str[j+k])
30 k++;
31 else
32 {
33 str[i+k] > str[j+k] ? i+=k+1 : j+=k+1;
34 if(i==j)
35 i++;
36 k=0;
37 }
38 }
39 return min(i,j);
40 }
41
42 char str[maxn];
43
44 int main()
45 {
```

```
46 int n;
47 cin >> n >> str;
48 for(int i=0;i<n;++i) str[i+n] = str[i];
49 cout << func1(str, n) << ' ' << func2(str, n) << endl;
```

假设输入均为整数加一个字符串，其中整数表示字符串的长度，整数不超过 100 000。

### ■ 判断题

22. 若输入 5 12312，则输出的第一个整数为 4。 ( )
23. 将第 12 行中的 while(k<n&&str[i+k]==str[j+k])修改为 while(str[i+k]==str[j+k])，则程序有可能会陷入死循环或出现数组越界。 ( )
24. 将第 35 行中的 i++修改为 j++，结果不会发生变化。 ( )
25. 在合法输入下，输出的两个整数一定是一样的。 ( )

### ■ 选择题

26. (4 分) 输入 9 321321323 时，第 33 行一共运行了 ( ) 次。
- A. 8 B. 7 C. 6 D. 5

27. (4 分) 假设字符串 s 是一个长度为 3 且只包含数字 0 到 9 的字符串，其中 s 的每个位置为 0 到 9 的可能性相同，则对该程序输入 3 和字符串 s 时，输出的第二个整数是 0 的概率为 ( )。

- A.  $\frac{1}{3}$  B.  $\frac{37}{100}$  C.  $\frac{17}{50}$  D.  $\frac{2}{5}$

(3)

```
01 #include<iostream>
02 #include<vector>
03 using namespace std;
04 const int maxn = 1e6+5;
05
06 vector<int> pri;
07 bool not_prime[maxn];
08 int value[maxn], num[maxn];
09
10 int init(int n)
11 {
12 value[1] = 1;
```

```
13 for (int i = 2; i <= n; ++i)
14 {
15 if (!not_prime[i])
16 {
17 pri.push_back(i);
18 value[i] = 2;
19 num[i] = 1;
20 }
21 for (int j : pri)
22 {
23 if (i * j > n) break;
24 not_prime[i * j] = true;
25 if (i % j == 0)
26 {
27 num[i * j] = num[i] + 1;
28 value[i * j] = value[i] / num[i * j] * (num[i * j] + 1);
29 break;
30 }
31 num[i * j] = 1;
32 value[i * j] = value[i] * 2;
33 }
34 }
35 return value[n];
36 }
37
38 int main()
39 {
40 int n;
41 cin >> n;
42 cout << init(n) << endl;
43 }
```

假设输入为一个不超过 1 000 000 的整数。

#### ■ 判断题

28. 输入 180 时, 输出结果为 18。 ( )
29. 分别输入 180 和 300 时, 在 main 函数退出前, 数组 num 在下标 1 到 100 上的值是不同的。 ( )
30. 输入 100 时, 在 main 函数退出前, vector 容器 pri 内的元素个数在 22 和 28 之间。 ( )

31. 将第 28 行修改为 `value[i * j] = value[i] * (num[i * j] + 1) / num[i * j]`, 结果不变。 ( )

■ 选择题

32. 该算法的时间复杂度为 ( )。
- A.  $O(n)$       B.  $O(n \log n)$       C.  $O(n \log \log n)$       D.  $O(\sqrt{n})$
33. (4 分) 数组 `value` 在下标  $i$  处的含义为 ( )。
- A.  $i$  的质因子个数      B.  $i$  的欧拉函数值
- C.  $i$  的所有因子和      D.  $i$  的因子个数

三、完善程序 (单选题, 每小题 3 分, 共计 30 分)

- (1) 有  $N$  种物品和一个容量为  $V$  的背包。

物品一共有 3 类:

第一类物品只能用 1 次 (0-1 背包);

第二类物品可以用无限次 (完全背包);

第三类物品最多只能用  $s_i$  次 (多重背包)。

每种物品的体积是  $v_i$ , 价值是  $w_i$ 。求解将哪些物品装入背包, 可使物品体积总和不超过背包容量, 且价值总和最大。输出最大价值。

输入格式:

第 1 行有两个整数  $N, V$ , 用空格隔开, 分别表示物品种数和背包容量。接下来有  $N$  行, 每行 3 个整数  $v_i, w_i, s_i$ , 用空格隔开, 分别表示第  $i$  种物品的体积、价值和数量。

$s_i = -1$  表示第  $i$  种物品只能用 1 次;

$s_i = 0$  表示第  $i$  种物品可以用无限次;

$s_i > 0$  表示第  $i$  种物品可以用  $s_i$  次。

输出格式:

输出一个整数, 表示最大价值。

数据范围:

$$0 < N, V \leq 1000$$

$$0 < v_i, w_i \leq 1000$$

$$-1 < s_i \leq 1000$$

输入样例:

```
4 5
1 2 -1
2 4 1
3 4 0
4 5 2
```

输出样例:

```
8
```

提示: 解决思路是将多重背包使用二进制拆分转换为 0-1 背包, 使得只包含 0-1 背包和完全背包。

```
01 #include <iostream>
02 #include <algorithm>
03 #include <vector>
04 using namespace std;
05 const int maxn = 2023;
06
07 int N, V;
08 int dp[maxn];
09 struct Item
10 {
11 int w, v;
12 int s;
13 };
14 void upd(int j, Item item)
15 {
16 dp[j] = max(dp[j], ①);
17 }
18 int main()
19 {
20 vector<Item> items;
21 cin >> N >> V;
22
23 for (int i = 1; i <= N; i++)
24 {
25 int w, v, s;
26 cin >> w >> v >> s;
27 if (②)
```

```
28 {
29 for (int k = 1; k <= s; k <= 1)
30 {
31 s -= k;
32 items.push_back({ ③ });
33 }
34 if (s > 0)
35 items.push_back({s * w, s * v, -1});
36 }
37 else
38 items.push_back({w, v, s});
39 }
40 for (auto item : items)
41 {
42 if (item.s == ④)
43 for (int j = V; j >= item.w; j--)
44 upd(j, item);
45 else
46 for (⑤)
47 upd(j, item);
48 }
49 cout << dp[V] << endl;
50 return 0;
51 }
```

34. ①处应填 ( )。

- A.  $dp[j - \text{item.w}] + \text{item.w}$       B.  $dp[j - \text{item.w}] + \text{item.v}$   
C.  $dp[j + \text{item.v}] + \text{item.w}$       D.  $dp[j + \text{item.w}] + \text{item.v}$

35. ②处应填 ( )。

- A.  $s > 0$       B.  $s == 0$       C.  $s == -1$       D.  $s != 0$

36. ③处应填 ( )。

- A.  $w * k, v * k, -1$       B.  $w * k, v, -1$   
C.  $w, v, -1$       D.  $w * k, v, 0$

37. ④处应填 ( )。

- A. 1      B. N      C. 1      D. -1

38. ⑤处应填 (        )。

- A. int j = item.w; j <= V; j--
- B. int j = item.v; j <= V; j++
- C. int j = item.w; j <= V; j ++
- D. int j = V; j>=item.w ; j--

(2) 给定  $n$  个正整数  $a_1, a_2, \dots, a_n$ ,  $m$  次询问, 每次询问一个给定的区间  $[l, r]$ , 输出  $a_l, a_{l+1}, \dots, a_r$  的最大公因数。

输入格式:

第 1 行两个整数  $n, m$ 。第 2 行  $n$  个整数表示  $a_1, a_2, \dots, a_n$ 。以下  $m$  行, 每行两个整数  $l, r$ , 表示询问区间的左右端点。

输出格式:

共  $m$  行, 每行表示一个询问的答案。

数据范围:

$1 \leq l \leq r \leq n \leq 100\,000$

$1 \leq m \leq 10^6$

$1 \leq a_i \leq 10^9$

输入样例:

5 3

4 12 3 6 7

1 3

2 3

5 5

输出样例:

1

3

7

```
01 #include <iostream>
02 #include <algorithm>
03 using namespace std;
04 const int maxn = 2e5 + 7;
05 const int maxp = 20;
06
07 int a[maxn], dp[maxn][maxp];
```

```
08 int gcd(int a, int b)
09 {
10 if (b == 0)
11 return a;
12 ①;
13 }
14 void init(int n)
15 {
16 for (int i = 1; i <= n; i++)
17 dp[i][②] = a[i];
18 for (int j = 1; (1 << j) <= n; j++)
19 for (int i = 1; i + (1 << j) - 1 <= n; i++)
20 dp[i][j] = gcd(dp[i][j - 1], dp[③][j - 1]);
21 }
22 int rmq(int l, int r)
23 {
24 int k = 0;
25 while (④)
26 k++;
27 return gcd(dp[l][k], dp[⑤][k]);
28 }
29 int main()
30 {
31 int n, m;
32 cin >> n >> m;
33 for (int i = 1; i <= n; i++)
34 cin >> a[i];
35 init(n);
36 for (int i = 1; i <= m; i++)
37 {
38 int l, r;
39 cin >> l >> r;
40 cout << rmq(l, r) << endl;
41 }
42 }
```

39. ①处应填 (     )。

- A. gcd(b, a%b)     B. gcd(b, a\*b)     C. gcd(a, a%b)     D. gcd(a, a/b)

40. ②处应填 (     )。

A. 1

B.  $\emptyset$

C. -1

D. i

41. ③处应填 (     )。

A.  $i+(1<<(j-1))-1$

B.  $i+(1<<j)-1$

C.  $i+(1<<j)$

D.  $i+(1<<(j-1))$

42. ④处应填 (     )。

A.  $(1<<k)<=r-l+1$

B.  $(1<<(k+1))<=r-l+1$

C.  $(k<<1)<=r-l+1$

D.  $((k+1)<<1)<=r-l+1$

43. ⑤处应填 (     )。

A.  $r-(1<<k)+1$

B.  $r-(1<<k)$

C.  $1+(1<<k)-1$

D.  $1+(1<<k)$

## 提高组 CSP-S 2024 初赛模拟卷 10

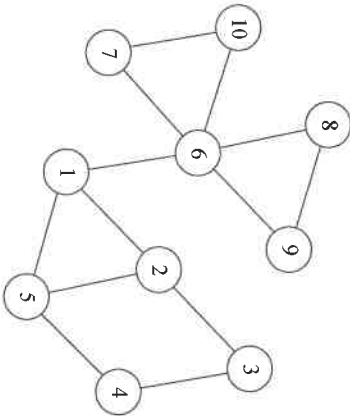
扫码获取  
答案

## 一、单项选择题（共 15 题，每题 2 分，共计 30 分；每题有且仅有一个正确选项）

1. 在 Linux 系统终端中，杀死名为 test 的进程的命令是（ ）。  
A. ps test      B. kill test      C. stop test      D. killall test
2. 机房的 IP 地址为 192.168.10.1/24，若想将其划分为 4 个子网，需要借的主机位数和每个子网的有效 IP 个数分别为（ ）。  
A. 1； 126      B. 1； 127      C. 2； 64      D. 2； 62
3. 下列语句中，可以用于清空 `stack<int> stk` 的是（ ）。  
A. `stk.clear()`;  
B. `while(stk.empty()) stk.pop()`;  
C. `while(stk.size()) stk.pop()`;  
D. `clear(stk)`;
4. 从 0~9 中选择 3 个数构成不含前导 0 且能被 3 整除的数的方案数为（ ）。  
A. 192      B. 256      C. 48      D. 228
5. 在一张无权图中使用广度优先搜索求从 1 号结点到其他所有结点的最短路径，下列说法中错误的是（ ）。  
A. 某一时刻，队列中某两个结点到结点 1 的距离的差为 0  
B. 某一时刻，队列中某两个结点到结点 1 的距离的差为 1  
C. 某一时刻，队列中某两个结点到结点 1 的距离的差为 2  
D. 某一时刻，队列中任意两个结点到结点 1 的距离差为 0
6. 关于图算法 BFS、Dijkstra 和 Bellman-Ford，求解边权均为 -1 的图的最长路径可以使用（ ）种算法，求解边权均为 1 的图的最长路径可以使用（ ）种算法，求解边权均小于 0 的图的最长路径可以使用（ ）种算法。（上述 3 个问题中若有不存在解的情况，所选算法需要判断不存在解的情况，只考虑正确性而不考虑效率。）  
A. 3； 1； 3      B. 1； 3； 1      C. 1； 2； 3      D. 3； 2； 1

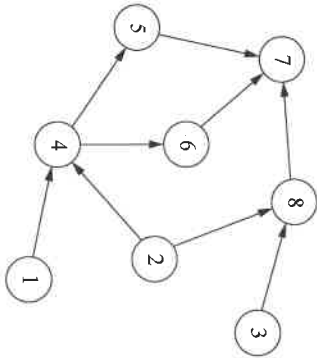
7. 在序列分块类问题中, 对长度为  $n$  的序列, 若取块长  $B$ , 零散块的修改复杂度为  $O(B)$ , 查询复杂度为  $O(\log^2 n)$ , 整块的修改复杂度为  $O(1)$ , 查询复杂度为  $O\left(\frac{n}{B} \log^2 n\right)$ , 则  $m$  次修改/查询的最坏复杂度最优为 ( )。
- A.  $O(m\sqrt{n} \log^2 n)$                       B.  $O(m\sqrt{n} \log n)$   
C.  $O(m\sqrt{n} \log n)$                       D.  $O(m\sqrt{n})$
8. 在 C++ 程序中执行如下代码片段后,  $t$  的值为 ( )。
- ```
int a = 1, b = 1, c = 4, d = 5, e = 1;  
bool f = true;  
auto t = ((a > b ^ c) && f) < d ^ e;
```
- A. true B. false C. 0 D. 1
9. 在一口无限深的井中, 蜗牛处于井口下方 5 个单位长度处, 每天白天以 $\frac{1}{2}$ 的概率向上爬 3 个单位长度, 晚上下滑 1 个单位长度, 则蜗牛爬出井口 (向上爬大于等于 5 个单位即为爬出井口) 的期望天数为 ()。
- A. $\frac{8}{3}$ B. $\frac{12}{5}$ C. 10 D. 5
10. 使用二进制分组优化多重背包问题, 数量为 10 的物品会被拆分为 () 件物品。
- A. 1 B. 2 C. 3 D. 4
11. 有一个数列定义如下: $a_0 = 1, a_1 = 1, a_n = (a_{n-1} + a_{n-2}) \bmod 10 (n > 1)$, 则 a_{1024} 和 a_{2048} 分别等于 ()。
- A. 3; 1 B. 5; 4 C. 8; 5 D. 1; 1
12. 下列代码片段中, 有 () 种方法可以判断状态 s 的第 x 个二进制位的情况 (0/1) (x 从 0 开始编号, 编号从低位到高位增大)。
- (1) `(bool)(s>>x&1)`
(2) `(bool)(s<<x&1)`
(3) `(~s>>x|~1)==s`
(4) `(~s|~1<x)==s`
- A. 1 B. 2 C. 3 D. 4

13. 下图中点双连通分量和边双连通分量分别有 () 个。



- A. 1; 1 B. 2; 3 C. 3; 2 D. 2; 2

14. 下图共有 () 种不同的拓扑序。



- A. 35 B. 21 C. 24 D. 76

15. 由 3 个中括号和 3 个小括号组成的本质不同的合法括号序列数为 (), 其中, 一对匹配的小括号之间不应有中括号。

- A. 770 B. 710 C. 192 D. 560

二、阅读程序 (程序输入不超过数组或字符串定义的范围; 判断题正确填√, 错误填×; 除特殊说明外, 判断题每题 1.5 分, 选择题每题 3 分, 共计 40 分)

(1)

```
01 #include <bits/stdc++.h>
02 using namespace std;
03 const int N = 2e5 + 10;
04
05 long long a[N];
06
```

```

07 int main() {
08     int n, m;
09     scanf("%d %d", &n, &m);    // 1 <= n, m <= 2e5
10     for(int i = 1; i <= n; ++ i) scanf("%d", a + i);
        // 1 <= a[i] <= 1e9
11     while(m --) {
12         int x;
13         scanf("%d", &x);    // 1 <= x <= 1e9
14         int lst = 1;
15         for(int i = 1; i <= n; ++ i) {
16             if(a[i] >= lst) {
17                 if(x <= a[i]) {
18                     a[i] += x - lst + 1;
19                     break;
20                 } else {
21                     int dis = a[i] - lst + 1;
22                     lst = a[i] + 1;
23                     a[i] += dis;
24
25                 }
26             }
27         }
28     }
29
30     for(int i = 1; i <= n; ++ i) printf("%ld\n", a[i]);
31     return 0;
32 }

```

假定输入数据范围： $1 \leq n, m \leq 2 \times 10^5, a_i, r \in [1, 10^9]$ 。

■ 判断题

16. 将第 5 行中的 long long 改为 int, 不影响程序结果。 ()
17. (2 分) 该算法的时间复杂度为 $O(nm)$ 。 ()
18. 当输入为 3 2 3 2 5 6 1 时, 输出为 7 2 7。 ()
19. 当输入为 3 2 3 2 1 6 6 时, 输出为 9 2 1。 ()

■ 选择题

20. 当输入为 4 5 10 20 30 40 20 10 30 25 5 时, 输出为 ()。
 - A. 10 20 30 40
 - B. 100 20 30 40
 - C. 30 30 60 100
 - D. 90 30 30 40

21. 当输入为 3 4 10 20 10 10 20 10 10 时, 输出为 ()。

- A. 10 20 60 B. 10 70 10 C. 60 20 10 D. 10 20 10

(2)

```
01 #include <bits/stdc++.h>
02
03 using namespace std;
04
05 int cow[300010];
06 int buf[300010], tot;
07 int buf2[300010], tot2;
08
09 signed main() {
10     int n;
11     scanf("%d", &n);
12     for(int i = 1; i <= n; ++ i) {
13         char ch;
14         for(ch = getchar(); ch != '0' && ch != '1'; ch = getchar());
15         cow[i] = ch - '0';
16     }
17     int tim = n << 1, start = 0;
18     for(int i = 1; i <= n + 1; ++ i) {
19         if(cow[i] == 1 && cow[i - 1] == 0) start = i;
20         else if(cow[i] == 0 && cow[i - 1] == 1) {
21             if(start == 1 || i == n + 1) {
22                 buf2[++ tot2] = i - start;
23                 tim = min(tim, (i - start - 1) << 1 | 1);
24             } else {
25                 buf[++ tot] = i - start;
26                 tim = min(tim, ((i - start - 1) >> 1) << 1 | 1);
27             }
28         }
29     }
30     long long ans = 0;
31     for(int i = 1; i <= tot; ++ i) ans += (buf[i] + tim - 1) / tim;
32     for(int i = 1; i <= tot2; ++ i) ans += (buf2[i] + tim - 1) / tim;
33     cout << ans << endl;
34     return 0;
35 }
```

输入为一个整数 n 以及一个长度为 n 的 01 字符串。保证 $1 \leq n \leq 3 \times 10^5$ 。

■ 判断题

22. 在第 18~29 行的循环结束后, 始终有 $\text{tot}+\text{tot2}=\text{n}$ 为 true。 ()
23. 若输入为 5 11111, 则输出为 1。 ()
24. 若输入为 6 011101, 则输出为 4。 ()

■ 选择题

25. 若输入为 9 101001101, 则 $\text{tot2}=(\quad)$ 。
A. 0 B. 1 C. 2 D. 3
26. 若输入为 15 110011100101101, 则 $\text{buf}[1..\text{tot}]$ 的内容为 ()。
A. 3 1 1 B. 3 1 2 C. 1 1 2 1 D. 1 1 2 2
27. 若输入为 13 0010110111011, 则 $\text{tot} + \text{tot2} = (\quad)$ 。
A. 1 B. 2 C. 3 D. 4

(3)

```
01 #include <bits/stdc++.h>
02 using namespace std;
03 const int N = 5e5 + 10;
04 int pls[N], cnt[N], dist[N], dist2[N];
05 signed main() {
06     ios::sync_with_stdio(false);
07     cin.tie(NULL);
08     int n, k;
09     cin >> n >> k;    // 1 <= k <= n <= 5e5
10     int x;
11     for(int i = 1; i <= k; ++ i) {
12         cin >> x;
13         pls[x] = i;
14         ++ cnt[x];
15     }
16     int ans = 0;
17     for(int i = 1; i <= k; ++ i) {
18         cin >> x;
19         ++ cnt[x];
20         if(pls[x]) {
21             int d;
```

```
22     if(pls[x] < i) {
23         d = k + pls[x] - i;
24     }
25     else d = pls[x] - i;
26     ans = max(ans, ++ dist[d]);
27     if(pls[x] < k + 1 - i) d = k + pls[x] - (k + 1 - i);
28     else d = pls[x] - (k + 1 - i);
29     ans = max(ans, ++ dist2[d]);
30 }
31 }
32 for(int i = 1; i <= n; ++ i)
33     if(!cnt[i])
34         ++ ans;
35     cout << ans << endl;
36     return 0;
37 }
```

假定输入数据中 $1 \leq k \leq n \leq 5 \times 10^5$ ，后跟的两个长度为 k 的序列分别是 $\{1, 2, \dots, n\}$ 的一个大小为 k 的子集的一个排列。

■ 判断题

28. 若输入为 6 3 1 2 3 2 3 1，则输出为 6。 ()
29. 若输入为 6 3 1 2 3 4 5 6，则输出为 1。 ()
30. (2 分) 若输入为 6 4 1 2 3 4 4 3 2 5，则执行到第 32 行时 cnt[1...n] 的值为 1 2 2 2 1 0。 ()

■ 选择题

31. 若输入为 6 4 1 2 3 4 4 3 1 5，则第 23 行执行 () 次。
- A. 1 B. 2 C. 3 D. 4
32. 若输入为 5 4 1 2 3 4 4 3 1 5，则输出为 ()。
- A. 1 B. 2 C. 3 D. 4
33. 若输入为 5 3 1 2 3 2 3 4，则执行完第 17~31 行的 for 循环后 dist2[2] = ()。
- A. 0 B. 1 C. 2 D. 3

三、完善程序（单选题，每小题 3 分，共计 30 分）

(1) (击靶) 在坐标轴上 $[-C, C]$ 区间内有 T 个靶子（不重合），射手刚开始处在坐标原点，进行 C 次操作。

L: 从坐标 x 处移动到坐标 $x-1$ 处。

R: 从坐标 x 处移动到坐标 $x+1$ 处。

F: 进行一次射击，若当前坐标处存在未被击倒的靶子，则击倒该靶子。

求对原操作序列修改一次后，最多能击倒多少靶子？

输入格式：

第 1 行是两个正整数 T 和 C 。第 2 行是 T 个正整数，表示靶子的坐标。第 3 行是一个长度为 C 的字符串，字典为 $\{L, R, F\}$ ，表示原操作序列。

解题思路：

从后往前枚举改变的具体时间和改变的操作类型。具体地说，对于同一种操作改变（例如，L 操作改成 R 操作），如果时间 x 之前的某个时间 y 发生了改变，则不管 y 多大，时间 x 所处的位置都是固定的一个值。可以预处理出这个位置，辅助变化位置的枚举。

```
01 #include <bits/stdc++.h>
02
03 using namespace std;
04
05 const int N = 1e5 + 10, DELTA = 1e5 + 1;
06 int pls[N], mp[N << 1], lft[N << 1], second[N << 1], LEFT[N << 1],
    nTar, nOp, curAns, ans;
07 char op[N];
08
09 void work(int delta, char from, char to) {
10     memset(second, 0, sizeof(second));
11     memcpy(lft, LEFT, sizeof(lft));
12     for( ① ) {
13         if( ② ) {
14             if(from == 'F' && mp[pls[i] + DELTA] && lft[pls[i] + DELTA]
                == i && second[pls[i] + DELTA] == 0)
15                 ans = max(ans, curAns - 1);
16             else if(to == 'F' && mp[pls[i] + DELTA] && lft[pls[i] +
                DELTA] == 0)
17                 ans = max(ans, curAns + 1);
```

```
18     else
19         ans = max(ans, curAns);
20     }
21     if(op[i] == 'F') {
22         if(mp[pls[i] + DELTA] && lft[pls[i] + DELTA] == i) {
23             lft[pls[i] + DELTA] = second[pls[i] + DELTA];
24             if(lft[pls[i] + DELTA] == 0)
25                 -- curAns;
26         }
27         pls[i] += delta;
28         if(mp[pls[i] + DELTA]) {
29             if(lft[pls[i] + DELTA] == 0) {
30                 ③;
31                 lft[pls[i] + DELTA] = i;
32             }
33             ④;
34             lft[pls[i] + DELTA] = min(lft[pls[i] + DELTA], i);
35         }
36         pls[i] -= delta;
37     }
38 }
39 }
40 }
41
42 signed main() {
43     ios::sync_with_stdio(false);
44     cin.tie(NULL);
45     cin >> nTar >> nOp; // nTar 表示靶子的个数, nOp 表示操作序列的长度
46     int tar;
47     for(int i = 1; i <= nTar; ++ i) {
48         cin >> tar; // 目标的位置
49         mp[tar + DELTA] = 1;
50     }
51     cin >> (op + 1); // 输入操作序列
52     int x = 0;
53     for(int i = 1; i <= nOp; ++ i) {
54         pls[i] = x;
55         if(op[i] == 'L') -- x;
56         else if(op[i] == 'R') ++ x;
57         else if(mp[x + DELTA] && !lft[x + DELTA]) {
58             ++ ans;
```

```
59         lft[x + DELTA] = i;  
60     }  
61     }  
62     int tmp = ans;  
63     memcpy(LEFT, lft, sizeof(lft));  
64     curAns = tmp; work(2, 'L', 'R');  
65     curAns = tmp; work(-2, 'R', 'L');  
66     curAns = tmp; work(1, 'L', 'F');  
67     curAns = tmp; work( ⑤ );  
68     curAns = tmp; work(-1, 'F', 'L');  
69     curAns = tmp; work(1, 'F', 'R');  
70     cout << ans << endl;  
71     return 0;  
72 }
```

34. ①处应填 ()。

- A. int i = 1; i <= nOP; ++ i B. int i = nOP; i >= 1; --i
C. int i = 1; i <= nTar; ++ i D. int i = nTar; i >= 1; -- i

35. ②处应填 ()。

- A. op[i] == to B. op[i] == from
C. op[i] == 'L' D. op[i] == 'R'

36. ③处应填 ()。

- A. ++ curAns B. -- curAns
C. ++ ans D. --ans

37. ④处应填 ()。

- A. LEFT[pls[i] + DELTA] = i
B. ans = curAns
C. second[pls[i] + DELTA] = i
D. ++ curAns

38. ⑤处应填 ()。

- A. 1, 'R', 'F' B. 2, 'R', 'F'
C. -1, 'R', 'L' D. -1, 'R', 'F'

(2) (字典序最小最长路径) 给出一张有 n 个结点、 m 条边的带权无环有向图, 边以 (a_i, b_i, l_i) 的形式给出, 表示从 a_i 到 b_i 的一条有向边, 边权为 l_i 。现在要求对于每个结点, 输出以它为起点且经过最多条边的路径经过的边数以及边权之和。如果有多条经过边数最多的路径, 则选择将路径边权按顺序排列, 得到序列中字典序最小的一条。

数据范围:

$$1 \leq n \leq 2 \times 10^5, \quad 1 \leq m \leq 4 \times 10^5, \quad 1 \leq a_i, b_i \leq 2 \times n, \quad 1 \leq l_i \leq 10^9.$$

输出格式:

第 1 行是两个正整数 n, m 。第 2 行到第 $n+1$ 行每行 3 个整数 a_i, b_i, l_i , 代表一条边。

解题思路:

输入为一个 DAG, 使用记忆化搜索来确定以某个结点为起点的最长且边权字典序最小的路径。在确定后继结点的选择时, 首先考虑路径长度最大, 而当多条路径的后继结点路径长度一样大时, 考虑将相同长度的结点按照(最小后继边权, 最优后继结点的哈希值)双关键字排序得到当前长度的结点的顺序, 作为当前结点的哈希值(也即结点在相同最大路径长度的结点中的排序)。这样在做后继结点的决策时, 只需要选择同深度结点中哈希值最小的结点即可。

```
01 #include <bits/stdc++.h>
02
03 using namespace std;
04
05 const int N = 2e5 + 10;
06 vector<pair<int, int>> g[N];
07 int dp[N], nexE[N], next[N], rk[N], id[N];
08 long long sum[N];
09 bool vis[N];
10
11 void work(int cur) {
12     if(vis[cur]) return ;
13     vis[cur] = true;
14     for(auto it : g[cur]) {
15         work(it.first);
16         dp[cur] = max(dp[cur], dp[it.first] + 1);
17     }
18     for(auto it : g[cur])
19         if(dp[it.first] + 1 == dp[cur] && (nexE[cur] == 0 || it.second
20             < nexE[cur]))
21             ①;
```

```
21 }
22
23 signed main() {
24     ios::sync_with_stdio(false);
25     cin.tie(NULL);
26     int n, m;
27     cin >> n >> m; // 输入结点数 n 和边数 m
28     int u, v, w;
29     for(int i = 1; i <= n; ++ i) {
30         cin >> u >> v >> w; // 从 u 到 v 的一条有向边, 边权为 w
31         gl[u].push_back(make_pair(v, w));
32     }
33     for(int i = 1; i <= n; ++ i)
34         if( ② )
35             work(i);
36     for(int i = 1; i <= n; ++ i)
37         id[i] = i;
38     sort(id + 1, id + n + 1, [δ](int x, int y) { return ③ ; });
39     vector<int> vec;
40     for(int i = 1; i <= n; ++ i) {
41         if(dp[id[i]] == 0) continue;
42         if(dp[id[i]] != dp[id[i - 1]]) δδ vec.size() {
43             sort(vec.begin(), vec.end(), [δ](int x, int y) {
44                 return nexE[x] < nexE[y] || ( ④ δδ rnK[nexT[x]]
45                     < rnK[nexT[y]]);
46             });
47             int cnt = 0;
48             for(int x : vec) rnK[x] = ++ cnt;
49             vec.clear();
50         }
51         vec.push_back(id[i]);
52         for(auto it : gl[id[i]])
53             if(dp[it.first] + 1 == dp[id[i]] δδ nexE[id[i]] == it.second
54                 δδ (nexT[id[i]] == 0 || rnK[nexT[id[i]]] > rnK[it.fi]))
55                 nexT[id[i]] = it.first;
56         sum[id[i]] += ⑤;
57     }
58     for(int i = 1; i <= n; ++ i)
59         cout << dp[i] << ' ' << sum[i] << endl;
60     return 0;
61 }
```

39. ①处应填 ()。

- A. `nexe[cur] = it.second` B. `nextE[cur] = dp[it.first] + 1`
C. `next[cur] = it.second` D. `next[cur] = it.first`

40. ②处应填 ()。

- A. `dp[i]` B. `!vis[i]` C. `!rnk[i]` D. `next[i]`

41. ③处应填 ()。

- A. `nexe[x] < nexe[y]` B. `next[x] < next[y]`
C. `dp[x] < dp[y]` D. `id[x] < id[y]`

42. ④处应填 ()。

- A. `rnk[x] == rnk[y]` B. `rnk[x] < rnk[y]`
C. `nexe[x] > nexe[y]` D. `nexe[x] == nexe[y]`

43. ⑤处应填 ()。

- A. `sum[id[i]] + nexe[id[i]]`
B. `sum[next[id[i]]] + nexe[id[i]]`
C. `sum[rnk[id[i]] + nexe[id[i]]`
D. `nexe[id[i]] + next[id[i]]`

